

НАЦІОНАЛЬНИЙ ТЕХНІЧНИЙ УНІВЕРСИТЕТ УКРАЇНИ
«КИЇВСЬКИЙ ПОЛІТЕХНІЧНИЙ ІНСТИТУТ імені ІГОРЯ СІКОРСЬКОГО»
Факультет інформатики та обчислювальної техніки
(повна назва інституту/факультету)

Кафедра інформатики та програмної інженерії
(повна назва кафедри)

«До захисту допущено»

Завідувач кафедри

_____ Едуард ЖАРІКОВ
(підпис) (ім'я прізвище)

“ ” _____ 2022 р.

Дипломний проєкт
на здобуття ступеня бакалавра
за освітньо-професійною програмою «Інженерія програмного забезпечення
комп'ютеризованих систем»
спеціальності «121 Інженерія програмного забезпечення»

на тему: Портал особистого кабінету співробітника сучасної компанії

Виконав студент IV курсу, групи ІТ-84
(шифр групи)

Трофімова Світлана Сергіївна _____
(прізвище, ім'я, по батькові) (підпис)

Керівник доцент, к.т.н., доц., Ліхоузова Т. А. _____
(посада, науковий ступінь, вчене звання, прізвище та ініціали) (підпис)

Консультант
з графічної
документації доцент, к.т.н., доц., Ліщук К. І. _____
(посада, науковий ступінь, вчене звання, прізвище та ініціали) (підпис)

Рецензент доцент кафедри ІСТ, к.т.н., доц., Ткач М. М. _____
(посада, науковий ступінь, вчене звання, прізвище та ініціали) (підпис)

Засвідчую, що у цій дипломній роботі
немає запозичень з праць інших
авторів без відповідних посилань.

Студент _____
(підпис)

Київ –2022

Національний технічний університет України
«Київський політехнічний інститут імені Ігоря Сікорського»
Факультет інформатики та обчислювальної техніки
Кафедра інформатики та програмної інженерії
Рівень вищої освіти – перший (бакалаврський)
Спеціальність – 121 Інженерія програмного забезпечення
Освітньо-професійна програма – Інженерія програмного забезпечення
комп'ютеризованих систем

ЗАТВЕРДЖУЮ

Завідувач кафедри

(підпис) Едуард ЖАРІКОВ
(ім'я прізвище)

“ ” _____ 2022 р.

ЗАВДАННЯ
на дипломний проєкт студенту

Трофімовій Світлані Сергіївні
(прізвище, ім'я, по батькові)

1. Тема проєкту Портал особистого кабінету співробітника сучасної компанії

керівник проєкту Ліхоузова Тетяна Анатоліївна, к.т.н., доцент
(прізвище, ім'я, по батькові, науковий ступінь, вчене звання)

затверджені наказом по університету від «10» червня 2022 р. №1033-с

2. Термін подання студентом проєкту « 19 » червня 2022 року

3. Вихідні дані до проєкту: технічне завдання

4. Зміст пояснювальної записки

1) Загальні положення: основні визначення та терміни, опис предметного середовища, огляд ринку програмних продуктів, постановка задачі.

2) Інформаційне забезпечення: вхідні дані, вихідні дані, опис структури бази даних.

3) Математичне забезпечення: змістовна та математична постановки задачі, обґрунтування та опис методу розв'язання.

4) Програмне та технічне забезпечення: засоби розробки, вимоги до технічного забезпечення, архітектура програмного забезпечення, побудова звітів.

5) Технологічний розділ: керівництво користувача, методика випробувань програмного продукту.

5. Перелік графічного матеріалу

1) Схема структурна варіантів використання HR-порталу _____

2) Схема взаємодії програми при вході у систему _____

3) Схема взаємодії програми при запиті на відпустку _____

6. Консультанти розділів проєкту

Розділ	Прізвище, ініціали та посада консультанта	Підпис, дата	
		завдання видав	завдання прийняв

7. Дата видачі завдання «10» березня 2022 року _____

Календарний план

№ з/п	Назва етапів виконання дипломного проєкту	Термін виконання етапів проєкту	Примітка
1	Вивчення рекомендованої літератури	10.03.2022	
2	Аналіз існуючих методів розв'язання задачі	01.04.2022	
3	Постановка та формалізація задачі	05.04.2022	
4	Розробка інформаційного забезпечення	15.04.2022	
5	Алгоритмізація задачі	20.04.2022	
6	Обґрунтування вибору використаних технічних засобів	01.05.2022	
7	Розробка програмного забезпечення	15.05.2022	
8	Налагодження програми	20.05.2022	
9	Виконання графічних документів	26.05.2022	
10	Оформлення пояснювальної записки	01.05.2022	
11	Подання ДП на попередній захист	06.06.2022	
12	Подання ДП рецензенту	12.06.2022	
13	Подання ДП на основний захист	19.06.2022	

Студент

(підпис)

Світлана ТРОФІМОВА

(ініціали, прізвище)

Керівник

(підпис)

Тетяна ЛІХОУЗОВА

(ініціали, прізвище)

АНОТАЦІЯ

Пояснювальна записка дипломного проєкту складається з чотирьох розділів, містить 39 таблиць, 10 рисунків та 10 джерел – загалом 62 сторінки.

Дипломний проєкт присвячений темі менеджменту людськими ресурсами.

Метою дипломного проєкту є розробка та збірка в одному місці інструментів для вирішення задач керування персоналом згідно основних задач HR.

Об'єкт дослідження: менеджмент людськими ресурсами.

Предмет дослідження: програмне забезпечення для вирішення задач керування персоналом.

У першому розділі наведено аналіз предметної області, аналіз відомих технічних рішень та аналіз потенційних конкурентів. На основі дослідження сформовано вимоги до проєкту. Другий розділ присвячений моделюванню та конструюванню програмного забезпечення, розробці архітектури, бази даних. Третій розділ містить опис аналізу якості програми та загальний опис етапів та процесів тестування. У четвертому розділі описано інструменту щодо розгортання та супроводу даного програмного забезпечення. Також наведено: опис програми, інструкцію користувача, програму та методику тестування, схему структурну варіантів використання HR-порталу, схему взаємодії програми при вході у систему, схему взаємодії програми при запиті на відпустку.

Результати роботи пройшли апробацію на XIII Міжнародній науково-практичній конференції «Modern directions of scientific research development» 15-17.06.2022, Чикаго, США та опубліковані в збірнику конференції с.342-347 за темою «Програмне забезпечення для вирішення задач керування персоналом».

КЛЮЧОВІ СЛОВА: ВЕБ-ДОДАТОК, ДОДАТОК УПРАВЛІННЯ ПЕРСОНАЛОМ, HRMS, ПРОГРАМНА СИСТЕМА, БАЗА ДАНИХ, NODE.JS, MONGODB.

ABSTRACT

The explanatory note of the diploma project consists of four sections, contains 39 tables, 10 figures and 10 sources - a total of 62 pages.

The diploma project is devoted to the topic of human resources management.

The purpose of the diploma project is to develop and assemble in one place tools for solving personnel management problems according to the main tasks of HR.

Object of research: human resource management.

Subject of research: the software for solving personnel management problems.

The first section presents an analysis of the subject area, analysis of known technical solutions and analysis of potential competitors. Based on the study, the requirements for the project are formed. The second section is devoted to modeling and design of software, architecture development, databases. The third section contains a description of the quality analysis of the program and a general description of the stages and processes of testing. The fourth section describes the tools for deploying and maintaining this software. Also provided: program description, user manual, program and testing methods, structural diagram of the options for using the HR-portal, the scheme of program interaction when logging in, the scheme of program interaction when requesting leave.

The results of the work were tested at the XIII International Scientific and Practical Conference "Modern Directions of Scientific Research Development" June 15-17, 2022, Chicago, USA and published in the conference proceedings p.342-347 on "Software for solving problems of personnel management".

KEYWORDS: WEB APPLICATION, PERSONNEL MANAGEMENT APP, HRMS, SOFTWARE SYSTEM, DATABASE, NODE.JS, MONGODB.

[illegible]

Факультет інформатики та обчислювальної техніки
Кафедра інформатики та програмної інженерії

“ЗАТВЕРДЖЕНО”

Завідувач кафедри

_____ Едуард ЖАРІКОВ

“ ____ ” _____ 2022 р.

**ПОРТАЛ ОСОБИСТОГО КАБІНЕТУ СПІВРОБІТНИКА СУЧАСНОЇ
КОМПАНІЇ**

Технічне завдання

КПІ.ІТ-8427.045440.01.91

“ПОГОДЖЕНО”

Керівник проєкту:

_____ Тетяна ЛІХОУЗОВА

Нормоконтроль:

_____ Катерина ЛІЩУК

Виконавець:

_____ Світлана ТРОФІМОВА

Київ – 2022

ЗМІСТ

1 НАЙМЕНУВАННЯ ТА ГАЛУЗЬ ЗАСТОСУВАННЯ	3
2 ПІДСТАВА ДЛЯ РОЗРОБКИ	4
3 ПРИЗНАЧЕННЯ РОЗРОБКИ	5
4 ВИМОГИ ДО ПРОГРАМНОГО ЗАБЕЗПЕЧЕННЯ	6
4.1 Вимоги до функціональних характеристик.....	6
4.2 Вимоги до надійності.....	6
4.3 Умови експлуатації	6
4.4 Вимоги до складу і параметрів технічних засобів.....	7
4.5 Вимоги до інформаційної та програмної сумісності.....	7
4.6 Вимоги до маркування та пакування	7
4.7 Вимоги до транспортування та зберігання.....	7
4.8 Спеціальні вимоги.....	7
5 ВИМОГИ ДО ПРОГРАМНОЇ ДОКУМЕНТАЦІЇ.....	9
6 СТАДІЇ І ЕТАПИ РОЗРОБКИ	11
7 ПОРЯДОК КОНТРОЛЮ ТА ПРИЙМАННЯ.....	12

1 НАЙМЕНУВАННЯ ТА ГАЛУЗЬ ЗАСТОСУВАННЯ

Назва розробки: Портал особистого кабінету співробітника сучасної компанії.

Галузь застосування: управління персоналом.

Наведене технічне завдання поширюється на розробку системи особистого кабінету співробітника (HR-портал), котра використовується для ефективного управління персоналом та призначена для використання менеджерами, адміністраторами, звичайними працівниками у будь-якій сучасній компанії.

					КПІ.IT-8427.045440.01.91	Арк.
Змін.	Арк.	№ докум.	Підп.	Дата.		3

2 ПІДСТАВА ДЛЯ РОЗРОБКИ

Підставою для розробки Порталу особистого кабінету співробітника є завдання на дипломне проєктування, затверджене кафедрою інформатики та програмної інженерії Національного технічного університету України «Київський політехнічний інститут імені Ігоря Сікорського».

					КПІ.ІТ-8427.045440.01.91	Арк.
Змін.	Арк.	№ докум.	Підп.	Дата.		4

3 ПРИЗНАЧЕННЯ РОЗРОБКИ

Розробка призначена для автоматизації ефективного управління персоналом.

Метою розробки є збірка в одному місці інструментів для вирішення задач керування персоналом згідно основних задач HR.

					КПІ.IT-8427.045440.01.91	Арк.
Змін.	Арк.	№ докум.	Підп.	Дата.		5

4 ВИМОГИ ДО ПРОГРАМНОГО ЗАБЕЗПЕЧЕННЯ

4.1 Вимоги до функціональних характеристик

4.1.1 Програмне забезпечення повинно забезпечувати виконання наступних основних функцій:

4.1.1.1 Для користувача:

- перегляд працівників компанії та особистих даних;
- подання заявки на відпустку;
- перегляд проєктів для співробітника;
- перегляд заробітної плати;
- відстежування відвідувань.

4.1.1.2 Для адміністратора системи:

- перегляд працівників компанії та додавання нових;
- погодження заявки на відпустку;
- корегування проєктів для співробітника;
- корегування заробітної плати;
- відстежування відвідувань.

4.1.2 Розробку виконати на платформі Windows 10.

4.1.3 Додаткові вимоги:

- реалізація аутентифікації користувачів;

4.2 Вимоги до надійності

4.2.1 Передбачити контроль введення інформації.

4.2.2 Передбачити захист від некоректних дій користувача.

4.2.3 Забезпечити цілісність інформації в базі даних.

					КПІ.IT-8427.045440.01.91	Арк.
Змін.	Арк.	№ докум.	Підп.	Дата.		6

4.3 Умови експлуатації

4.3.1 Умови експлуатації згідно СанПін 2.2.2.542 – 96.

4.3.2 Обслуговування

Не висуваються.

4.3.3 Обслуговуючий персонал

Не висуваються.

4.4 Вимоги до складу і параметрів технічних засобів

4.4.1 Програмне забезпечення повинно функціонувати на серверах та комп'ютерах з операційною системою Windows/Ubuntu/MacOS.

4.4.2 Мінімальна конфігурація технічних засобів:

4.4.2.1 Тип процесору Intel Core i3.

4.4.2.2 Об'єм ОЗП 4 ГБ.

4.5 Вимоги до інформаційної та програмної сумісності

4.5.1 Програмне забезпечення повинно працювати під управлінням операційних си-стем сімейства WIN32 (Windows'XP, Windows NT і т.д.) або Unix.

4.5.2 Програмне забезпечення повинно забезпечувати доступ до веб-додатку.

4.6 Вимоги до маркування та пакування

Вимоги до маркування та пакування не пред'являються.

4.7 Вимоги до транспортування та зберігання

Вимоги до транспортування та зберігання не пред'являються.

					КПІ.ІТ-8427.045440.01.91	Арк.
Змін.	Арк.	№ докум.	Підп.	Дата.		7

4.8 Спеціальні вимоги

Згенерувати установчу версію програмного забезпечення.

					КПІ.ІТ-8427.045440.01.91	Арк.
						8
Змін.	Арк.	№ докум.	Підп.	Дата.		

5 ВИМОГИ ДО ПРОГРАМНОЇ ДОКУМЕНТАЦІЇ

5.1 Програмні модулі, котрі розробляються, повинні бути задокументовані, тобто тексти програм повинні містити всі необхідні коментарі.

5.2 Програмне забезпечення повинно мати довідникову систему.

5.3 У склад супроводжувальної документації повинні входити наступні документи:

5.3.1 Пояснювальна записка не менше ніж на 50 аркушах формату А4 (без додатків 5.3.2 - 5.3.6).

5.3.2 Технічне завдання.

5.3.3 Керівництво користувача.

5.3.4 Керівництво адміністратора

5.3.5 Програма та методика тестування

5.4 Графічна частина повинна бути виконана не менше ніж на 3 аркушах формату А3, котрі включаються у якості додатків до пояснювальної записки:

5.4.1 Схема структурна варіантів використання HR-порталу.

5.4.2 Схема взаємодії програми при вході у систему.

5.4.3 Схема взаємодії програми при запиті на відпустку.

6 СТАДІЇ І ЕТАПИ РОЗРОБКИ

Таблиця 6.1 – Стадії і етапи розробки

№	Назва етапу	Строк	Звітність
1.	Вивчення рекомендованої літератури	10.03.2022	
2.	Аналіз існуючих методів розв'язання задачі	01.04.2022	
3.	Постановка та формалізація задачі	05.04.2022	Технічне завдання
4.	Розробка інформаційного забезпечення	15.04.2022	Схема структурна програмного забезпечення
5.	Алгоритмізація задачі	20.04.2022	
6.	Обґрунтування вибору використаних технічних засобів	01.05.2022	
7.	Розробка програмного забезпечення	15.05.2022	Тексти програмного забезпечення
8.	Налагодження програми	20.05.2022	Програма та методика тестування
9.	Виконання графічних документів	26.05.2022	Графічний матеріал проекту
10.	Оформлення пояснювальної записки	01.05.2022	Пояснювальна записка проекту
11.	Подання ДП на попередній захист	06.06.2022	
12.	Подання ДП рецензенту	12.06.2022	
13.	Подання ДП на основний захист	19.06.2022	

7 ПОРЯДОК КОНТРОЛЮ ТА ПРИЙМАННЯ

Тестування розробленого програмного продукту виконується відповідно до “Програми та методики тестування”.

					КПІ.ІТ-8427.045440.01.91	Арк.
						11
Змін.	Арк.	№ докум.	Підп.	Дата.		

**Пояснювальна записка
до дипломного проєкту**

на тему: Портал особистого кабінету співробітника сучасної компанії

КПІ.IT-8427.045440.02.81

Київ – 2022

ЗМІСТ

ВСТУП	15
1 АНАЛІЗ ВИМОГ ДО ПРОГРАМНОГО ЗАБЕЗПЕЧЕННЯ	17
1.1 Загальні положення	17
1.2 Змістовний опис і аналіз предметної області	18
1.3 Аналіз успішних ІТ-проектів.....	20
1.4 Аналіз вимог до програмного забезпечення	29
1.5 Постановка задачі	40
Висновки до розділу	41
2 МОДЕЛЮВАННЯ ТА КОНСТРУЮВАННЯ ПРОГРАМНОГО ЗАБЕЗПЕЧЕННЯ	42
2.1 Моделювання та аналіз програмного забезпечення.....	42
2.2 Архітектура програмного забезпечення.....	42
2.3 Конструювання програмного забезпечення.....	48
2.4 Аналіз безпеки даних	49
Висновки до розділу	52
3 АНАЛІЗ ЯКОСТІ ТА ТЕСТУВАННЯ ПРОГРАМНОГО ЗАБЕЗПЕЧЕННЯ	54
3.1 Аналіз якості ПЗ.....	54
3.2 Опис процесів тестування.....	59
3.3 Опис контрольного прикладу	65
Висновки до розділу	72
4 ВПРОВАДЖЕННЯ ТА СУПРОВІД ПРОГРАМНОГО ЗАБЕЗПЕЧЕННЯ	73
4.1 Розгортання програмного забезпечення.....	73
4.2 Робота з програмним забезпеченням	74
Висновки до розділу	74
ВИСНОВКИ.....	75
СПИСОК ВИКОРИСТАНИХ ДЖЕРЕЛ.....	76
ДОДАТКИ.....	77

ПЕРЕЛІК УМОВНИХ ПОЗНАЧЕНЬ

IDE	– Integrated Development Environment – інтегроване середовище розробки
API	– Application programming interface, прикладний програмний Інтерфейс
SDK	– Software development kit
IT	– Інформаційні технології
ER	– Entity-Relation diagram
OC	– Операційна система
БД	– База даних

ВСТУП

Сучасний стан економічного зростання вимагає кваліфікованого управління людськими ресурсами. Робота зі співробітниками є обов'язковою частиною ефективної роботи будь-якої сучасної компанії. Те, що рухає бізнесом, — це злагоджені дії та рішення групи професіоналів-однодумців. Досягнення компанії забезпечують працівники, а не теорії складенні експертами. Співробітники створюють додану цінність, забезпечують параметри якості, виконують основну працю для досягнення поставлених цілей і представляють компанію на ринку. Комунікацію зі співробітниками усіх рівнів налаштовує HR-менеджер, праця якого є багатофункціональна. Він укладає тон всій організації, відповідає за налагодження і підтримку цілого бізнесу і безпосередньо відповідає за роботу співробітників. Ефективність праці вимагає знання, а також осмислення політики і напрямків компанії, можливостей, досвіду та навичок. Тому виникає потреба спростити та підвищити продуктивність HR-відділів, що дозволить усім співробітникам побачити свій прогрес і можливості.

Світові тенденції показують, що нині web-додатки для HR-спеціалістів з кожним роком стають все більш популярним в ІТ світі. Системи управління людськими ресурсами, або скорочено HRMS, використовуються організаціями будь-якого розміру для підтримки різних функцій HR.

Перелік найкращих HRMS у світі:

- Monday.com. Найкраще підходить для створення індивідуальних кадрових і робочих процесів;
- Sapling. Найкращий HRMS для середніх компаній (50-2000 співробітників);
- Eddy. Найкраще для легкої адаптації;
- Sage HRMS. Найкращий HRMS для малого бізнесу;
- PeopleStreme. Найкраще програмне забезпечення HRMS.

Керівники відділу кадрів і персонал є основними користувачами, враховуючи, що вони керують повсякденними роботами з робочою силою і відповідають за відповідність вимогам та звітність про ефективність. Однак HR — не єдиний відділ, який отримує вигоду. Компанії можуть надати менеджерам і співробітникам можливість самообслуговування для виконання спільних завдань, що є важливою ознакою продажу для молодших працівників. Керівники можуть використовувати HRMS для генерування даних про тенденції персоналу та їх наслідки для бізнесу.

Враховуючи, що витрати, пов'язані з кадрами, є одними з найбільших витрат компанії, інтеграція HRMS із системою бухгалтерського обліку є неоціненною для фінансових команд. Провідні постачальники вийдуть за рамки простого бухгалтерського обліку, щоб допомогти компанії витягнути більше фінансових даних із даних кадрів.

Мета проєкту: розробити та зібрати в одному місці інструменти для вирішення задач керування персоналом згідно основних задач HR. Планується розробити web-додаток, у якому буде реалізовано перегляд основної інформації про співробітника та керівника, а також можливість подачі заяв задля покращення основних аспектів роботи HR-співробітника та керівника відділу/компанії.

					КПІ.IT-8427.045440.02.81	Арк.
Змін.	Арк.	№ докум.	Підп.	Дата.		
						16

1 АНАЛІЗ ВИМОГ ДО ПРОГРАМНОГО ЗАБЕЗПЕЧЕННЯ

1.1 Загальні положення

HRMS, або система управління людськими ресурсами, — це набір програмних додатків, які використовуються для управління людськими ресурсами та пов'язаними процесами протягом усього життєвого циклу співробітника. HRMS дає змогу компанії повністю зрозуміти свою робочу силу, дотримуючись зміненого податкового законодавства та трудового законодавства.

Система управління людськими ресурсами (HRMS) — це тип інформаційної системи (IC), яка призначена для управління комп'ютеризованими й автоматизованими процесами людських ресурсів (HR) організації. Це поєднання апаратних і програмних ресурсів, які розміщують і надають більшість, якщо не всю, бізнес-логіку відділу кадрів.

HRMS розгортається на сервері додатків, який забезпечує внутрішній та/або віддалений доступ всьому уповноваженому персоналу. Як частина автономної системи або системи планування ресурсів підприємства (ERP), HRMS залежить від програмного забезпечення для управління персоналом, яке інтегроване зі спеціальними для персоналу бізнес-процесами та функціями, які дають співробітникам відділу кадрів можливість виконувати рутинні операції, такі як керування записами співробітників, нарахування заробітної плати, управління відвідуваністю та оцінювання ефективності. Кожна функція може бути доступна як частина основного HRMS або додана як програмні модулі/компоненти.

У більшості середовищ HRMS інтегрований і пов'язаний з іншими допоміжними системами, такими як відстеження часу, відвідуваність, фінанси/рахунки та адміністрування.

1.2 Змістовний опис і аналіз предметної області

Людські ресурси – це ті, хто складає робочу силу певної організації, економіки чи сектору бізнесу. Термін «людський капітал» можна розуміти як синонім терміну «людські ресурси». Але в той же час під «людським капіталом» зазвичай розуміється більш вузький погляд, як-от знання, які втілюють люди, а також економічне зростання. Деякі інші терміни, які іноді можуть використовуватися замість терміну «людські ресурси», включають «таланти», «співробітники», «працівники», «люди» або «персонал» [1].

Очікується, що будь-який відділ кадрів певної організації здійснюватиме управління людськими ресурсами, наглядаючи за різними аспектами працевлаштування (наприклад, дотримання трудового законодавства та деяких стандартів зайнятості), деякими аспектами звільнення та прийому на роботу, а також адміністрацією пільги працівникам.

Відомо, що будь-який менеджер з персоналу відповідає за визначення робочих потреб персоналу, використовує тимчасовий персонал або наймає працівників для задоволення таких потреб, підбирає та навчає кращих працівників, контролює їх роботу, керує відносинами між працівниками, нарахування заробітної плати, пільг і компенсацій, підготовка особистої політики та обліку працівників, забезпечення високої продуктивності, забезпечення рівних можливостей для всіх співробітників, вирішення питань дискримінації та інших питань продуктивності.

Будь-яка компанія з співробітниками матиме в тій чи іншій формі Систему управління персоналом, починаючи від ручних документів і електронних таблиць до комплексних хмарних систем управління персоналом, призначених для обробки всіх кадрових процесів.

Систему управління персоналом використовують фахівці з персоналу, керівники та співробітники[5]. Керування стандартними виплатами, такими як страхування здоров'я, стоматології, життя та непрацездатності, пенсійні

плани, відшкодування вартості навчання та гнучкі рахунки витрат із пільгою податковою діяльністю є давньою функцією основного відділу кадрів.

Багато компаній перенесли ці функції зі своїх локальних комп'ютерів до хмарних постачальників. Нові платформи медичного страхування та оздоровчі портали виходять за рамки основ охорони здоров'я, щоб забезпечити онлайн-доступ до постачальників, медичних порад і фітнес-спільнот.

За словами Стейсі Харріс, аналітика з HR-технологій, купуючи технологію пільг, більшість зосередиться на партнерстві з постачальниками пільг, а не на програмному забезпеченні, яке використовується для їх адміністрування. «Я рідко приймаю рішення щодо вибору переваг лише на основі досвіду користувачів, як я міг би зробити з управлінням талантами або основним HRMS», — сказала Харріс.

Успішні компанії не мають часу витратити на ручне виконання повторюваних кадрових завдань. У міру того, як ваша компанія росте, вам потрібно шукати рішення, які допоможуть вам подолати проблеми з персоналом, які стримують вас. Тут на допомогу приходить програмне забезпечення системи управління людськими ресурсами (HRMS). Зазвичай HRMS вирішує завдання пошуку кандидатів, також виконує адміністрування співробітниками, допомагає вести облік нараховувань заробітної плати, адміністрування доступних пільг, облік табелів робочого часу, а також вирішує завдання по керуванню продуктивністю [2].

Програмне забезпечення HR дає можливість організаціям оцифрувати задачі ведення записів, обчислень та комунікації, які виконує відділ кадрів, і розподілити деякі з цих обов'язків по всій організації. Такі функції, як облік робочого часу, нарахування заробітної плати, підбір персоналу, відповідність нормативним вимогам і адміністрування пільг, можна перенести з паперу та керувати — зазвичай більш ефективно — на комп'ютерах. Ця часткова автоматизація ручних завдань може знизити витрати на оплату праці, оптимізувати HR-процеси та зробити їх більш ефективними.

Однак ефект від технології виходить далеко за межі цих звичайних переваг автоматизації HR. Коли більше співробітників працюють краще і повністю реалізують свій потенціал, результатом часто є покращення креативності та продуктивності, що в кінцевому підсумку призводить до більшого прибутку.

Інвестування в програмне забезпечення HRMS дозволяє підприємствам упорядкувати й автоматизувати трудомісткі й виснажливі завдання, щоб HR-команди могли зосередитися на діяльності з більш високою доданою вартістю.

1.3 Аналіз успішних ІТ-проектів

У 1970-х роках, коли компанії намагалися автоматизувати управління своїми людьми, нарахування заробітної плати стало першою функцією HRMS, яка була комп'ютеризована. Але для обчислення заробітку працівника, утримання відрахувань, друку паперового чека та відстеження зобов'язань із заробітної плати знадобилася мейнфреймова технологія. З впровадженням ERP-систем у світ HR підхід до питання змінився. SAP R/2 був запущений в 1979 році, інтегрувавши функціональні можливості HR разом в одну базу даних ERP з плануванням виробництва, управлінням матеріалами та фінансами. Комбінований доступ до даних через мережу мейнфрейма в реальному часі привернув велику увагу і спонукав більше компаній вийти на ринок. Але лише на початку 2000-х років із широким поширенням прямих депозитів і самообслуговування співробітників процес нарахування заробітної плати став повністю електронним[6].

Наступною важливою віхою в історії HRMS стало впровадження PeopleSoft у 1987 році, ERP, яка була побудована на середовищі клієнт-сервер, а не на мейнфреймі.

PeopleSoft представляв зміну в сфері HRMS, оскільки це було спеціально створене програмне забезпечення, створене з нуля та кероване вимогами HR-бізнесу, а не доповненням до фінансової ERP.

					КПІ.ІТ-8427.045440.02.81	Арк. 20
Змін.	Арк.	№ докум.	Підп.	Дата.		

Наприкінці 1980-х років PeopleSoft був одним із перших, хто запровадив більш повну систему HRMS[9]. Окрім нарахування заробітної плати, він пропонував ведення обліку співробітників, набір, робочий час і відвідування, адміністрування пільг, винагороду, звітність про відповідність та інші функції, які допомагають спеціалістам з управління персоналом автоматизувати більшу частину життєвого циклу співробітників і приймати кращі рішення щодо персоналу.

Ринок HRMS розпочав свою історію спочатку 90-х років минулого століття, тоді постачальники HRMS ERP, такі як JD Edwards і Oracle, представили HR ERP, які були розроблені, щоб охопити всі функції HR, включаючи основні HR, набір, навчання, нарахування заробітної плати в США та звітність[10].

Піднесення Інтернету наприкінці 1990-х принесло переваги автоматизації ще більшій кількості HR-процесів. Наприклад, паперові оголошення про довідку були замінені електронними дошками вакансій, що дало рекрутерам і кандидатам нові способи підключення.

У 90-х роках минулого століття відбулась значуща подія для всього світу - перехід від підходу розробки клієнт-сервер типу до Інтернету, інновація, яка позбулася необхідності встановлювати програмне забезпечення локально.

Таким чином, це відкриває системи HRMS для користувачів поза кадрами, таких як менеджери та співробітники, відкриваючи двері для самообслуговування співробітників HRMS. У той же час наступне покоління систем управління персоналом стало більш спеціалізованим, індустрія побачила початок «найкращого в своєму класі» HRMS, тому програма HR, наприклад Taleo, зосереджена лише на підборі персоналу, а не на «одне рішення для усіх задач» ERP-система.

У 2010-х роках хмарні технології були основними — тепер HR-команди в компаніях будь-якого розміру могли дозволити собі набір додатків, не вкладаючи коштів у дороге комп'ютерне обладнання чи ІТ-персонал для

роботи та обслуговування системи. Сфера HRMS продовжує розвиватися, оскільки постачальники надають нові опції та функціональні можливості для задоволення зростаючих потреб HR.

1.3.1 Аналіз відомих технічних рішень

Програмне забезпечення HR зазвичай постачається у вигляді набору модулів, кожен з яких розроблений для певного HR-процесу.

Найбільш повні платформи мають кілька назв:

- система управління людськими ресурсами (HRMS);
- інформаційна система людських ресурсів (HRIS);
- управління людським капіталом (HCM).

Терміни часто використовуються як взаємозамінні, але є деякі помітні відмінності між HRIS та HRMS та HCM. Коротше кажучи, HRIS та HRMS є майже синонімами фактичних програмних платформ, які ви можете придбати, тоді як HCM — це зонтичний термін для процесів та програмного забезпечення HR.

Ці комплексні пакети програмного забезпечення HR зазвичай включають такі програмні модулі, які часто можна придбати окремо:

- основний кадровий склад (виплати, документи співробітників, нарахування заробітної плати тощо);
- управління талантами (компенсація, навчання, результативність, найм, спадкоємність);
- управління персоналом (планування, робочий час і відвідуваність);
- надання послуг (самообслуговування співробітників і керівників).

Суспільні тенденції, посилені пандемією, справляють великий вплив на кадрові процеси та програмне забезпечення в тандемі з новими технологіями, які починають проникати в бізнес-процеси. Ось сім великих тенденцій.

– Штучний інтелект (ШІ). Штучний інтелект може виявляти закономірності, вибирати з-поміж альтернатив і робити прогнози — іншими словами, усе те, що люди роблять, підбираючи, оцінюючи та розвиваючи працівників. Автоматизація цих процесів за допомогою ШІ може зробити їх більш точними та ефективними. Більшість постачальників програмного забезпечення для персоналу агресивно додають можливості штучного інтелекту, але є серйозні побоювання, що ця технологія іноді може посилити, а не усунути расизм, сексизм та інші упередження. Компанії також стурбовані тим, що залишають важливі рішення за технологічною «чорною скринькою».

– Доповнена реальність (AR) і віртуальна реальність (VR). AR накладає цифрову інформацію на реальне середовище, в той час як VR намагається цифрово моделювати все середовище. Обидва тільки починають трансформувати корпоративне навчання, надаючи віртуальний практичний досвід із інструментами та процедурами, які співробітники повинні вивчати для своєї роботи. Використання AR та VR у навчанні, орієнтованому на HR, може допомогти запобігти кризі знань від масового виходу на пенсію бейбумерів.

– Блокчейн. Технологія баз даних для розподілу записів по ланцюжку комп'ютерів, блокчейн робить дані по суті постійними та несприйнятливими до злому. Біткойн — найвідоміший блокчейн-додаток, але компанії усвідомили потенціал для обміну записами співробітників та іншою інформацією з персоналу. Початкове використання технології блокчейн HR включає безпечну передачу документації з нарахування заробітної плати, облікових даних кандидатів і трудових контрактів.

– Різноманітність та включення. Програми по боротьбі з дискримінаційною практикою працевлаштування та індивідуальною поведінкою сягають 1960-х років, але протести Black Lives Matter 2020 року були тим, що нарешті підштовхнуло до корпоративної розплати. Постачальники програмного забезпечення для персоналу відповіли новими

					КПІ.IT-8427.045440.02.81	Арк.
Змін.	Арк.	№ докум.	Підп.	Дата.		
						23

функціями D&I, включаючи спеціалізований навчальний контент та інструменти штучного інтелекту, призначені для усунення упередженості при прийомі на роботу.

– Психічне здоров'я. Боротьба працівників із ізоляцією та стресом через карантин через COVID-19 та очікування віддаленої роботи викликали новий інтерес до підтримки психічного здоров'я нарівні з фізичним здоров'ям. Роботодавці почали надавати програми та портали для психічного здоров'я у своїх пакетах пільг.

– Дистанційна та гібридна робота. Блокування через COVID-19 змусили більшість службовців працювати з дому. Коли обмеження почали знімати, компанії досліджували нову гібридну модель, яка поєднує будинок і офіс. Фізична розсіяність робочої сили викликала попит на програмне забезпечення для відеоконференцій, інструменти для командної співпраці та інші гібридні технології на робочому місці. Постачальники програмного забезпечення для персоналу відповіли новими функціями для віддаленого керування продуктивністю та підбору персоналу, залучення співробітників, моніторингу та здоров'я, а також для керування протоколами безпеки COVID-19.

– Роботизована автоматизація процесів (RPA). Імітуючи дії людини, «боти» RPA можуть швидше та точніше виконувати звичайні процеси управління персоналом, такі як вибір переваг, відшкодування витрат та сортування резюме. Останніми роками більшість постачальників включили RPA до продуктів HR.

1.3.2 Аналіз відомих програмних продуктів

Що ми шукаємо, коли вибираємо, які інструменти входять до остаточного списку HRMS? Ось короткий виклад моїх критеріїв оцінки:

– Інтерфейс користувача (UI): повинен бути простим та привабливим. Знаходження всіх типових функцій HR повинно бути легким.

КПІ.IT-8427.045440.02.81					Арк.
Змін.	Арк.	№ докум.	Підп.	Дата.	24

Всі персональні дані повинні бути чіткими, гарно організованими та швидко доступними для пошуку[4].

– Зручність використання: повинно бути легко навчитися та опанувати. Для системи необхідна хороша технічна підтримка, також підтримка користувачів, наявності навчальні посібники та модулі з навчання.

– Характеристики та функціональність:

○ Масштабованість та гнучкість. Програмне забезпечення має розвиватися та розширюватися разом із вами, оскільки потреби будь-якої компанії та персоналу змінюються пропорційно з часом.

○ Технічні засоби залучення – при виборі HRMS необхідно дивитися за межі «типових» функцій HR, аби отримати інструменти для залучення співробітників, вимірювання задоволеності та розвитку продуктивності. Вбудована система управління навчанням також є значним показником.

○ Табельний облік, ведення звітності заробітної плати, підрахунок податків – інструмент має обробляти всі стандартні елементи роботи відділу кадрів.

○ Розділ звітів та аналітичні дані. Програмне забезпечення має забезпечувати практичну інформацію в реальному часі, використовуючи дані, які надаються при роботі. Необхідна властивість налаштовування звітів, їх треба легко експортувати.

– Інтеграції: чи легко підключитися до інших інструментів? Будь-які попередньо вбудовані інтеграції? Додаток має підключатися до будь-яких дошок вакансій для спрощеного набору кандидатів.

– Вартість програмного забезпечення: Ціна має відповідати функціям, можливостям та варіантам використання. Ціноутворення має бути чіткими, гнучкими та прозорими.

Далі представлений короткий опис варіантів систем управління персоналом, 5 найкращих з них:

					КПІ.IT-8427.045440.02.81	Арк.
Змін.	Арк.	№ докум.	Підп.	Дата.		25

Monday.com – найкраще підходить для створення індивідуальних кадрових і робочих процесів. Це платформа HRMS, яка дозволяє HR-командам налаштовувати та запускати всіх людей і робочі процеси за допомогою технології перетягування. Інструмент також містить різноманітні шаблони, призначені для використання з працівниками. Інструмент дозволяє менеджерам з найму стежити за процесом найму та відстежувати кандидатів за допомогою вбудованих форм, які автоматично записують всю відповідну контактну інформацію. Щойно кандидата наймуть, члени команди можуть оновити статус своєї заявки, щоб відстежувати всі вакансії в режимі реального часу.

Monday.com також містить готовий спеціальний шаблон, який містить усі уроки та читання, які знадобляться новачкові протягом перших кількох тижнів. Спеціальні шаблони також використовують автоматизацію без коду, щоб сповіщати відповідних членів команди про їхні завдання, щоб нових найнятих правильно вітали. Ще одна чудова функція – трекер відпусток, який дозволяє співробітникам планувати час відпустки.

Monday.com коштує від 8 \$ за користувача/місяць. Також доступна 14-денна безкоштовна пробна версія.

Sapling – Найкращий HRMS для середніх компаній (50-2000 співробітників). Sapling — це платформа для співпраці з людьми, найкраще підходить для компаній середнього рівня з розкиданими міжнародними офісами. Інструмент автоматизує робочі процеси, щоб увімкнути, відхилити та використовувати людські дані в існуючих системах.

Sapling дозволяє користувачам створювати автоматизовані робочі процеси для введення тексту та налаштування. Члени команди можуть автоматично призначати робочі процеси людей залежно від місця розташування, відділу та статусу зайнятості. Sapling також дозволяє членам команди зберігати дані про людей і керувати ними, а також створювати організаційні діаграми, а кінцеві користувачі можуть створювати профілі.

Sapling пропонує інтеграцію з різноманітним іншим програмним забезпеченням, включаючи системи відстеження заявників, програмне забезпечення для нарахування заробітної плати, програмне забезпечення для відстеження робочого часу тощо. Інструмент також має API та веб-хуки для подальших параметрів інтеграції.

Модулі адаптації розсади починаються з 4,92 доларів США.

Eddy – найкраще підходить для зручності. Eddy — це комплексний HR-пакет, розроблений для місцевих підприємств для оптимізації важких кадрових процесів і підвищення продуктивності співробітників. Заснована в 2017 році підприємцем, який став професійним баскетболістом Тревісом Хансеном, компанія Eddy швидко розвивалася завдяки своїм простим, але потужним продуктам, високоякісному обслуговуванню клієнтів і місцевим підприємствам, зосередженим на безробітній робочій силі.

Завдяки Eddy підприємства можуть наймати, вмикати, керувати та платити співробітникам за допомогою одного простого у використанні програмного забезпечення.

Основні функції включають керування оголошеннями про вакансію, відстеження кандидатів на роботу та складну систему відстеження кандидатів (ATS). Система адаптації дозволяє менеджерам з персоналу створювати власні пакети адаптації, призначати завдання та легко збирати цифрові підписи.

Основною функцією Eddy є управління людьми - користувачі можуть отримати доступ до каталогу компанії та зберігати важливі файли співробітників, записи про результати роботи, навчання та сертифікати у файлах співробітників. Ви також можете створювати власні політики РТО та затверджувати або відхиляти запити; за допомогою інструмента відстеження часу співробітникам буде легко вмикати та вимикати час. Переглядайте процес нарахування заробітної плати за допомогою внутрішньої системи Eddy з повним набором послуг, яка включає підтримку кількох періодів платежів та підготовку, подачу та розповсюдження W-2 наприкінці року.

					КПІ.IT-8427.045440.02.81	Арк.
Змін.	Арк.	№ докум.	Підп.	Дата.		27

Eddy ціна починається від 8 доларів США на співробітника на місяць з безкоштовною пробною версією.

Sage HRMS – Найкращий HRMS для малого бізнесу. Sage відомий тим, що керує бухгалтерським обліком, людськими ресурсами, платежами, активами, будівництвом, нерухомістю та корпоративними системами. Вони пропонують хмарні рішення локально або обидва. Їхні налаштовані рішення можуть бути адаптовані як для стартапів і масштабних компаній, так і для корпоративних компаній.

Програмне забезпечення чудово підходить для навчання завдяки різноманітним модулям, розробленим для допомоги кожному унікальному процесу управління людьми, наприклад, підбору, навчання, підвищення зарплати, зміна роботи тощо.

Sage надає повний список доповнень, які можна використовувати для інтеграції різних рішень у свій робочий процес HRMS.

Sage HRMS часто віддає перевагу інтеграційним рішенням від першої сторони, які можуть поставити користувачів у не вигідне становище, якщо вони мають інші інструменти для управління проектами або нарахування зарплатної плати, які потрібно підключити. Через це вони втрачають кілька балів в оцінці ансамблю.

Sage починається від 10 доларів США за користувача на місяць з безкоштовною пробною версією.

PeopleStreme – Найкраще програмне забезпечення системи управління персоналом. PeopleStreme був розроблений Ascender, міжнародною компанією, яка управляє людським капіталом і програмним забезпеченням для нарахування зарплатної плати, а також спеціалізується на розробці технологій для підтримки програм управління персоналом. Його використовують такі бренди, як Audi та Movember Foundation.

PeopleStreme чудово справляється з критеріями оцінки зручності, оскільки програмне забезпечення просте у використанні з інтуїтивно

зрозумілою системою приладної панелі. Крім того, вони можуть швидко вирішити проблеми підтримки, коли це необхідно.

Інтеграції включають інші інструменти Ascender, GO1, Австралійський інститут людських ресурсів (AHRI), Broadbean, CT Management, Indeed, Intercheck, SEEK і Workpro.

Одним із недоліків є те, що дані може бути трохи важко оцінити на перший погляд, оскільки внутрішні кандидати а) ідентифікуються лише за номером, а не по імені, і б) не відображаються у випадку дублікатів (тобто кандидатів, які подають заявки на кілька ролей в компанії).

PeopleStreme пропонує ціни за запитом і має безкоштовну пробну версію.

1.4 Аналіз вимог до програмного забезпечення

Розроблена програма повинна мати наступний функціонал

- перегляд працівників компанії та додавання нових;
- подання заявки на відпустку, та погодження менеджером;
- корегування проєктів для співробітника;
- корегування заробітної плати;
- відстежування відвідувань.

При аналізі предметної області та технологічних рішень було виявлено три ролі задля забезпечення повного функціоналу програми:

Працівник – сутність звичайного користувача, який може переглядати інформацію про себе та створити запит на відпустку або відмітити присутність.

Менеджер – сутність користувача, який має аккаунт в системі та має роль начальника у відділі, він може переглядати працівників, які належать до його департаменту та налаштовувати усі данні.

Адміністратор – сутність користувача, який має роль супер користувача у системі. Він має доступ до усього функціоналу системи.

Портал – суть вашого існуючого програмного забезпечення, для якого стає задачею реалізувати функціонал HR-порталу.

Розроблене програмне забезпечення повинно реалізовувати наступні функції:

- аутентифікація користувачів;
- перегляд працівників;
- сортування та пошук працівників за будь-якими критеріями;
- додавання працівника;
- корегування даних працівника;
- створення запиту на відпустку;
- погодження/відхилення запиту на відпустку;
- додавання та перегляд відвідувань робочого місця.

Визначено наступні варіанти використання:

Таблиця 1.1 – Варіант використання UC-1

Унікальний ідентифікатор	UC-1
Назва	Аутентифікація користувачів
Опис	Користувач входить у систему
Учасники	Користувач
Передумови	Відкрита веб-сторінка логіну, користувач має кредити для входу
Постумови	Користувач успішно/неуспішно ввійшов у систему
Сценарій	– Користувач вводить дані логіну – Користувач вводить дані паролю – Натиснення кнопки «увійти»

Таблиця 1.2 – Варіант використання УС-2

Унікальний ідентифікатор	УС-2
Назва	Перегляд працівників
Опис	Користувач переглядає усіх наявних працівників
Учасники	Користувач
Передумови	Успішний вхід у систему, визначена роль адміністратору або менеджера
Постумови	Відображений список працівників
Сценарій	Користувач натискає кнопку «Усі працівники»

Таблиця 1.3 – Варіант використання УС-3

Унікальний ідентифікатор	УС-3
Назва	Сортування та пошук працівників за будь-якими критеріями
Опис	Користувач сортує та знаходить працівників за різними критеріями
Учасники	Користувач
Передумови	Успішний вхід у систему, визначена роль адміністратору або менеджера, користувач зайшов на сторінку перегляду працівників
Постумови	Відсортовано або знайдено усіх працівників, які відповідають критерію у запиті
Сценарій	Користувач натискає кнопку пошуку під критерієм у таблиці співробітників, вводить туди данні та отримує перелік

Таблиця 1.4 – Варіант використання УС-4

Унікальний ідентифікатор	УС-4
Назва	Додавання працівника
Опис	Користувач додає дані про працівника
Учасники	Користувач
Передумови	Успішний вхід у систему, визначена роль адміністратору або менеджера
Постумови	Додані данні нового працівника
Сценарій	Користувач натискає кнопку «Додати працівника», Вводить всі дані, натискає кнопку зберегти

Таблиця 1.5 – Варіант використання УС-5

Унікальний ідентифікатор	УС-5
Назва	Корегування даних працівника
Опис	Користувач корегує дані працівника
Учасники	Користувач
Передумови	Успішний вхід у систему, визначена роль адміністратору або менеджера
Постумови	Дані працівника змінено
Сценарій	Користувач натискає кнопку «Корегувати дані працівника», Змінює необхідні дані, натискає кнопку зберегти

Таблиця 1.6 – Варіант використання УС-6

Унікальний ідентифікатор	УС-6
Назва	Створення запиту на відпустку
Опис	Користувач створює запит на відпустку
Учасники	Користувач
Передумови	Успішний вхід у систему
Постумови	Додано новий запит на відпустку
Сценарій	Користувач натискає кнопку «Запит на відпустку», заповнює необхідні дані, натискає кнопку «Зберегти»

Таблиця 1.7 – Варіант використання УС-7

Унікальний ідентифікатор	УС-7
Назва	Погодження/відхилення запиту на відпустку
Опис	Користувач отримує новий запит на погодження та перевіряє правильність даних
Учасники	Користувач
Передумови	Успішний вхід у систему, визначена роль адміністратору або менеджера, знайдено новий запит на погодження
Постумови	Погоджено/відхилено запит на відпустку
Сценарій	Користувач натискає кнопку «Запити на відпустку», перевіряє вірність усіх даних, змінює статус заявки, натискає кнопку «Зберегти»

Таблиця 1.8 – Варіант використання УС-8

Унікальний ідентифікатор	УС-8
Назва	Додавання та перегляд відвідувань робочого місця
Опис	На екрані відображені відвідування за місяць
Учасники	Користувач
Передумови	Успішний вхід у систему
Постумови	Відмічено/переглянуто присутність
Сценарій	Користувач натискає кнопку «Відмітити присутність»/ «Переглянути присутність»

Таблиця 1.9 – Варіант використання УС-9

Унікальний ідентифікатор	УС-9
Назва	Додавання та корегування /перегляд даних про проєкт
Опис	На екрані відображені дані про проєкт
Учасники	Користувач
Передумови	Успішний вхід у систему
Постумови	Переглянуто/відкориговано дані про проєкт
Сценарій	Користувач натискає кнопку «Коригувати проєкт»/ «Переглянути проєкт»

Таблиця 1.10 – Варіант використання UC-10

Унікальний ідентифікатор	UC-10
Назва	Додавання та перегляд даних про зарплату
Опис	На екрані відображені дані про зарплату
Учасники	Користувач
Передумови	Успішний вхід у систему
Постумови	Переглянуто/відкориговано дані про зарплату
Сценарій	Користувач натискає кнопку «Корегувати зарплати»/ «Переглянути зарплату»

1.4.1 Розроблення функціональних вимог

Найпоширенішим і дуже зручним способом опису функціональних вимог є діаграма варіантів використання (Use Case diagram). Варіанти використання системи зображені на рисунку 1.1

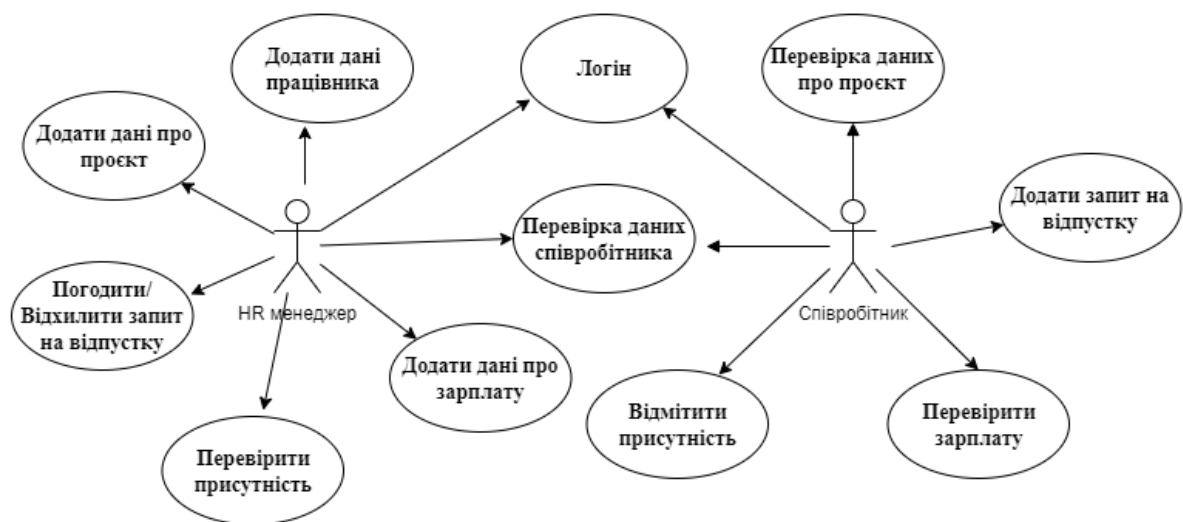


Рисунок 1.1 – Схема структурна варіантів використання HR-порталу

Опис функціональних вимог деталізовано за допомогою таблиць 1.11 – 1.21.

Таблиця 1.11 – Функціональна вимога RQ-1

Унікальний ідентифікатор	RQ-1
Назва	Логін у систему
Опис	Система має надавати можливість логіну.

Таблиця 1.12 – Функціональна вимога RQ-2

Унікальний ідентифікатор	RQ-2
Назва	Перевірка даних співробітника
Опис	Система має надавати можливість перевірки усіх даних співробітника

Таблиця 1.13 – Функціональна вимога RQ-3

Унікальний ідентифікатор	RQ-3
Назва	Додання даних про зарплату
Опис	Система повинна надавати можливість додавати дані про зарплату співробітника

Таблиця 1.14 – Функціональна вимога RQ-4

Унікальний ідентифікатор	RQ-4
Назва	Перевірка присутності
Опис	У системі повинна бути передбачена можливість перевіряти присутність співробітника на робочому місці

Таблиця 1.15 – Функціональна вимога RQ-5

Унікальний ідентифікатор	RQ-5
Назва	Погодження/Відхилення запиту на відпустку
Опис	У системі має бути передбачена можливість погодження/відхилення запиту на відпустку

Таблиця 1.16 – Функціональна вимога RQ-6

Унікальний ідентифікатор	RQ-6
Назва	Додати дані про проєкт
Опис	Має бути передбачена можливість додання даних про проєкт

Таблиця 1.17 – Функціональна вимога RQ-7

Унікальний ідентифікатор	RQ-7
Назва	Додати дані працівника
Опис	У системі передбачена можливість додавати дані співробітника

Таблиця 1.18 – Функціональна вимога RQ-8

Унікальний ідентифікатор	RQ-8
Назва	Перевірка даних про проєкт
Опис	Має бути перевірка співробітником даних про проєкт

Таблиця 1.19 – Функціональна вимога RQ-9

Унікальний ідентифікатор	RQ-9
Назва	Додати запит на відпустку
Опис	Співробітник може додавати запит на відпустку

Таблиця 1.20 – Функціональна вимога RQ-10

Унікальний ідентифікатор	RQ-10
Назва	Перевірити зарплату
Опис	Користувач має можливість перевірити заробітну плату

Таблиця 1.21 – Функціональна вимога RQ-11

Унікальний ідентифікатор	RQ-11
Назва	Відмітити присутність
Опис	Для користувача передбачена можливість відмічати присутність на робочому місці

Таблиця 1.22 – Матриця трасування вимог і варіантів використання

	RQ-1	RQ-2	RQ-3	RQ-4	RQ-5	RQ-6	RQ-7	RQ-8	RQ-9	RQ-10	RQ-11
UC-1											
UC-2											
UC-3											
UC-4											
UC-5											
UC-6											
UC-7											
UC-8											
UC-9											
UC-10											

1.4.2 Розроблення нефункціональних вимог

Важливо, щоб продукт працював належним чином і мав відмінні риси. З нефункціональними вимогами можна створити продукт з унікальними властивостями.

Продукт повинен відповідати вимогам:

- швидкість;
- безпечність;
- портативність;
- сумісність;
- ємність;
- надійність.

1.5 Постановка задачі

Основна мета розробки проєкту – це зібрати в одному місці інструменти для вирішення задач керування персоналом згідно основних задач HR. Необхідно розробити web-додаток, у якому буде реалізовано перегляд основної інформації про співробітника та керівника, а також можливість подачі заяв задля покращення основних аспектів роботи HR-співробітника та керівника відділу/компанії. Головним модулем розробки є модернізація процесу подачі, погодження або відхилення заявок на відпустки.

Програма буде призначеною для менеджерів з управління персоналом, відділу кадрів невеликих компаній, які за допомогою цієї розробки зможуть спростити та автоматизувати свою працю.

Реалізація програми використовує 3 основні програмні ролі: адміністратор, менеджер, працівник.

Стоїть задача виконати реалізацію повного функціоналу програми:

- аутентифікація користувачів;
- перегляд працівників;
- сортування та пошук працівників за будь-якими критеріями;

					КПІ.IT-8427.045440.02.81	Арк. 40
Змін.	Арк.	№ докум.	Підп.	Дата.		

- додавання працівника;
- корегування даних працівника;
- створення запиту на відпустку;
- погодження/відхилення запиту на відпустку;
- додавання та перегляд відвідувань робочого місця.

Висновки до розділу

Створення високопродуктивних систем з вимогами до масштабованості, гнучкості та стійкості у сучасності є досить розповсюдженим. Для розробки HRMS у наш час виділяється доволі велика ніша задач. Тому у даному розділі були розглянуті усі можливості та найкращі практики розробки систем менеджменту співробітниками.

У цьому розділі аналізуються існуючі технічні рішення, вимоги до програмного забезпечення та демонстраційні проекти. Вимоги представлені за допомогою структурної діаграми прецедентних, функціональних і нефункціональних вимог.

					КПІ.IT-8427.045440.02.81	Арк.
Змін.	Арк.	№ докум.	Підп.	Дата.		41

2 МОДЕЛЮВАННЯ ТА КОНСТРУЮВАННЯ ПРОГРАМНОГО ЗАБЕЗПЕЧЕННЯ

2.1 Моделювання та аналіз програмного забезпечення

Основним модулем застосунку є Запит на відпустку. Для визначення процесу необхідно використати нотацію опису бізнес-процесів – BPMN. Bpmn діаграма представлена на Рисунку 2.1.

Користувач створює заявку на відпустку, система перевіряє чи соціальна ця відпустка. Залежно від типу відпуски надсилається на погодження до керівника, або до HR-менеджеру компанії. Після погодження відбувається перевірка дат заявки, якщо дати відповідають умовам, то створюється наказ на відпустку та процес завершується.

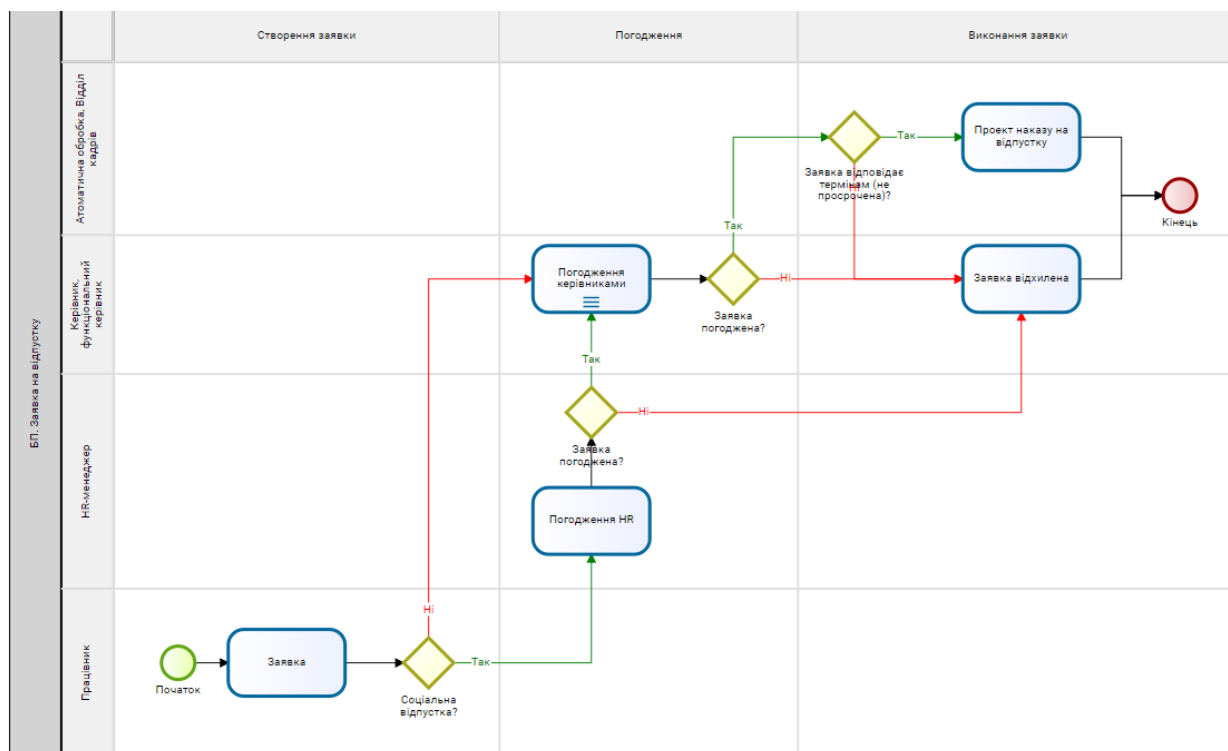


Рисунок 2.1 – Схема бізнес-процесу

2.2 Архітектура програмного забезпечення

Архітектура програмного забезпечення - це просто організація системи. Організація включає в себе всі компоненти того, як вони взаємодіють один з

одним, середовище, в якому вони працюють, і принципи, які використовуються для розробки програмного забезпечення. У багатьох випадках це також може включати еволюцію програмного забезпечення в майбутнє.

Для опису архітектур можна використовувати діаграми діяльності. Діаграма діяльності описує, як діяльність координується для надання послуг, які можуть бути на різних рівнях абстракції. Часто подія має бути реалізована якоюсь дією, особливо якщо дія має на меті досягти багатьох різних речей, які потрібно скоординувати, або тому що події у варіанті використання взаємопов'язані, особливо випадки використання, коли дії можуть перетинатися і їх потрібно координувати. Він також підходить для моделювання того, як набір параметрів використання координується для представлення робочого процесу. На малюнку 2.2 показана діаграма діяльності процесу входу в систему.

Іншим важливим аспектом у побудові архітектури програмного забезпечення є аналіз процесів. Аналіз процесів – це процес постійного вдосконалення, коли організації аналізують те, як вони роблять речі, щоб знайти більш ефективні методи для виконання певного завдання. Цей аналіз базується на трьох елементах будь-якої діяльності: вхід, процес і вихід. Аналіз процесу має справу з тим, як вхідні дані перетворюються в бажаний результат. Однією з цілей цього аналізу є скорочення кількості ресурсів, включаючи час, що використовуються для отримання необхідного результату[3].

Аналітик процесу буде спостерігати та документувати, як виконується конкретне завдання, щоб детально описати всі етапи та людей, які в ньому беруть участь, а потім він зазвичай розробляє блок-схему, яка ілюструє те, як вхідні дані протікають через процес. Іншою метою аналізу процесів є стандартизація діяльності таким чином, щоб кожен, хто має базові знання про компанію та діяльність, що виконується, міг ефективно внести свою частину процесу. Це також допомагає організаціям легше навчати людей.

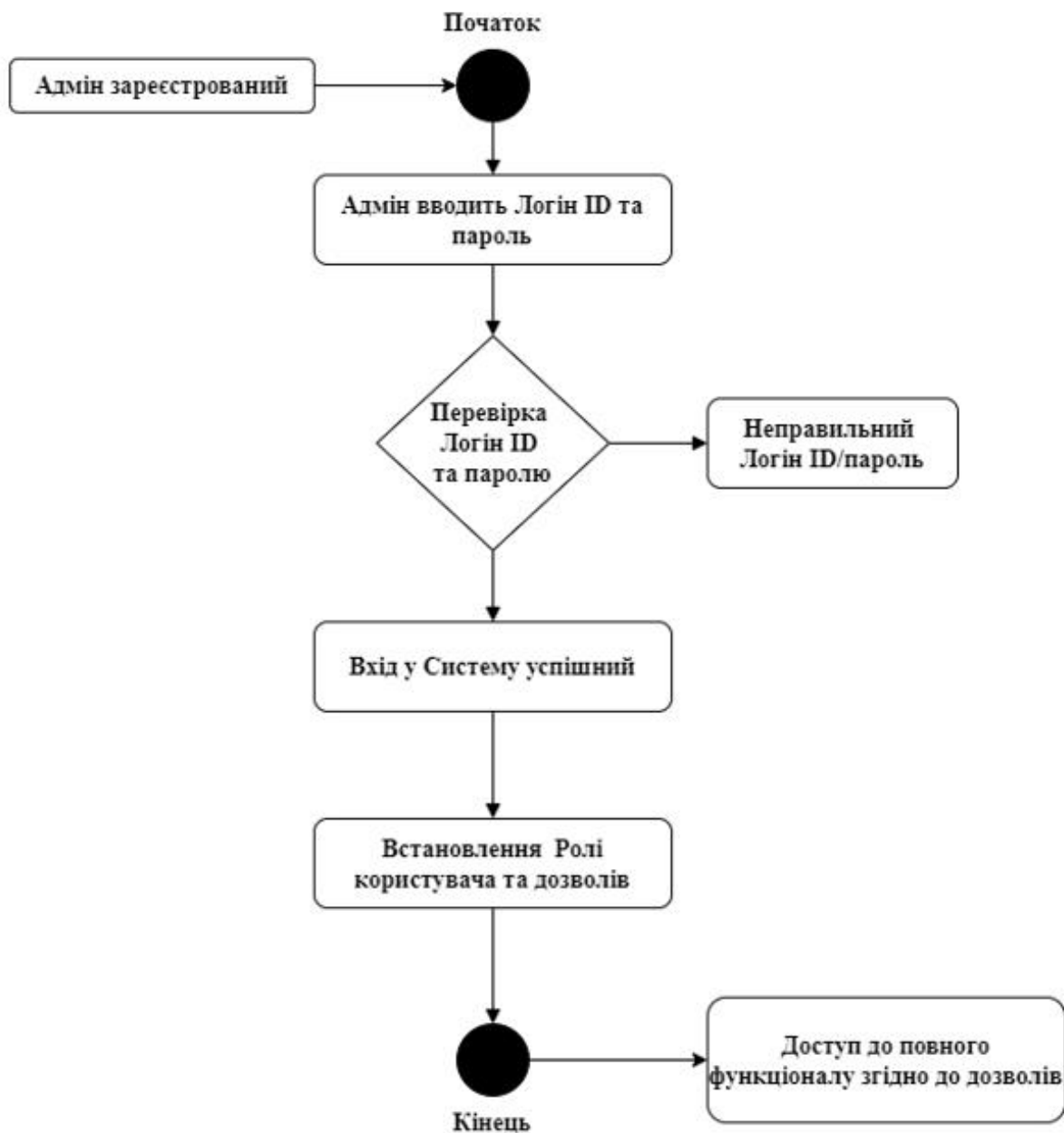


Рисунок 2.2 – Схема взаємодії програми при вході у систему

Основним модулем розробки є створення запиту на відпустку. На рисунку 2.3 можна побачити аналіз процесу.



Рисунок 2.3 – Схема взаємодії програми при запиті на відпустку

Для вирішення поставлених задач було створені колекції у MongoDB, схеми яких представлені на рисунку 2.4.

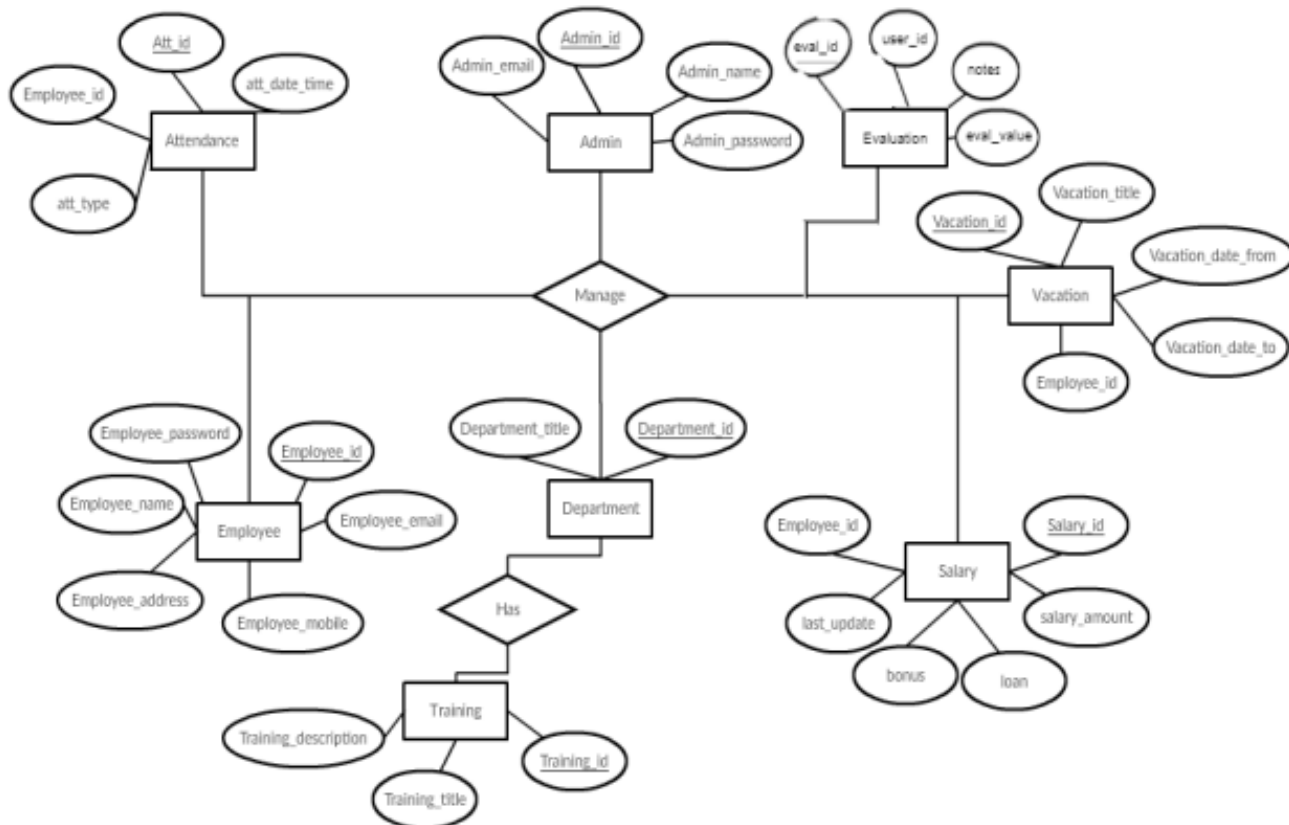


Рисунок 2.4 – ER-діаграма для HR-порталу

Далі представлені описи сутностей бази даних.

Таблиця 2.1 – Сутність користувача

Атрибут	Тип
type	String
email	mongoose.SchemaTypes.Email
password	String
name	String
dateOfBirth	Date
contactNumber	String
department	String
Skills	String
dateAdded	Date
designation	String

Таблиця 2.2 – Сутність зарплати

Атрибут	Тип
accountManagerID	Schema.Types.ObjectId
employeeID	Schema.Types.ObjectId
salary	Number
bonus	Number
reasonForBonus	String

Таблиця 2.3 – Сутність проєкту

Атрибут	Тип
employeeID	Schema.Types.ObjectId
title	String
type	String
status	String
description	String
startDate	Date
endDate	Date

Таблиця 2.4 – Сутність відпустки

Атрибут	Тип
applicantID	Schema.Types.ObjectId
title	String
type	String
startDate	Date
endDate	Date
appliedDate	Date
period	Number
reason	String
adminResponse	String

2.3 Конструювання програмного забезпечення

Для реалізації поставленої задачі створення HR-порталу будемо використовувати Node Express, який є фреймворком веб-додатків для Node.js, а інтерфейс створений за допомогою HTML, Bootstrap, CSS та JS. У якості бази даних використовується MongoDB.

Node JS – це інструмент JavaScript з відкритим кодом, створений на основі JavaScript Engine Google Chrome, але чи є Node JS Framework? Він використовується для створення масштабованих мережевих додатків за допомогою керованої подіями, неблокуючої моделі вводу-виводу, що робить його швидким і економним. Це дозволяє розробникам використовувати одну мову (JavaScript) як на стороні сервера, так і на стороні клієнта. Це неминуче покращує те, наскільки ефективно та швидко можуть працювати програми, одночасно зменшуючи час їх розробки та витрати на обслуговування[7].

Хоча більшість людей, які чули про Node JS, знають, які його основні види використання, не всі знають напевно, до якої категорії віднести цю технологію. Багато людей вважають Node JS фреймворком, а інші вважають, що це мова програмування.

Express — це мінімальний і гнучкий фреймворк веб-додатків Node.js, який надає потужний набір функцій для розробки веб- та мобільних додатків. Це сприяє швидкому розвитку веб-додатків на основі вузлів. Ось деякі ключові особливості фреймворку Express.

- Дозволяє налаштувати проміжне програмне забезпечення для відповіді на HTTP-запити.
- Визначити таблиці маршрутизації для виконання різних дій на основі методів HTTP та URL-адрес.
- Дозволяє динамічно відображати сторінки HTML на основі передачі параметрів шаблонам.

MongoDB — це документно-орієнтована база даних NoSQL для зберігання великих обсягів даних. Замість використання таблиць і рядків, як у

традиційних реляційних базах даних, MongoDB використовує колекції та документи [8]. Документ складається з пар ключ-значення, які є основною одиницею даних у MongoDB. Колекції містять набори документів і функцій, які еквівалентні таблицям реляційної бази даних. MongoDB — це база даних, яка з'явилася в середині 00-х років нашого століття.

Особливості MongoDB[8].

- Кожна база даних містить колекції, які в свою чергу містять документи. Кожен документ може мати різну кількість полів. Розмір і зміст кожного документа можуть відрізнятися один від одного.
- Документація більш структурована відповідно до того, як розробники структурують свої класи та об'єкти відповідною мовою програмування. Розробники часто кажуть, що їхні класи не є рядками і стовпцями, а мають чітку структуру пар ключ-значення.
- Рядок (або документ, на який посилається в MongoDB) не повинен мати попередньо визначену схему. Натомість ви можете створювати поля динамічно.
- Модель даних, доступна в MongoDB, дозволяє легше представляти ієрархічні зв'язки, зберігати масиви та інші складніші структури.
- Масштабованість. Середовища MongoDB дуже масштабовані. Компанії по всьому світу визначили кластери, деякі з яких працюють із понад 100 вузлами, з приблизно мільйоном документів у базі даних.

Основні методи функціоналу системи наведено у таблиці 2.5.

2.4 Аналіз безпеки даних

Величезні набори даних, які можуть зібрати наші все більш складні інструменти безпеки, здатні ідентифікувати потенційні загрози ІТ-безпеці та допомогти нам краще захищати системи компанії. Але є проблема: як нам визначити, оцінити та діяти на основі точних правильних наборів даних, тих, які допоможуть у вирішенні найважливіших проблем безпеки?

Таблиця 2.5 – Основні методи функціоналу

Функція (викликаний Path)	Опис
/login	Вхід у систему
/logout	Вихід із системи
/view-profile	Перегляд профіля
/view-all-employees	Перегляд усіх співробітників
/add-employee	Додати співробітника
/all-employee-projects/:id	Переглянути проекти усіх співробітників
/leave-applications	Перегляд заявок на відпустку
/employee-profile/:id	Перегляд профілю співробітника за id
/edit-employee/:id	Коректування профілю співробітника за id
/edit-employee-project/:id	Коректування проекту співробітника за id
/add-employee-project/:id	Додавання проекту співробітника за id
/employee-project-info/:id	Перегляд інформації про проект співробітника за id
/view-attendance	Перегляд відвідувань
/mark-attendance	Проставити відвідування за сьогодні
isLoggedIn(req, res, next)	перевірка на доступність чи увійшов користувач. Відбувається редірект на home сторінку
notLoggedIn(req, res, next)	перевірка на доступність чи увійшов користувач. Відбувається редірект на login сторінку

В основі безпеки, керованої даними, лежать методи аналітики, здатні просіяти інформацію, яку створюють журнали мережі та пристроїв безпеки. Платформа або служба аналітики безпеки збирає та аналізує дані з таких джерел, як інформація про безпеку та системи керування подіями, щоб забезпечити видимість потенційних загроз безпеці.

Останні події, що впливають на аналітику безпеки, включають поширення Інтернету речей; Можливості машинного навчання та автоматизація також впливають. Як наслідок, для проведення аналітики безпеки на основі даних необхідні більш складні засоби. Набори даних з кожним днем збільшуються, що ускладнює пошук і докази, скажімо, того, що користувач робить щось підозріле. Неоціненні вибачення можуть занадто легко загубитися в прислів'ї сіна; Як обробляти глобальні набори корпоративних даних зараз є вирішальними знаннями.

Людьми керує багато різних речей – голод любові чи прийняття, чи, можливо, слави чи грошей. Але майже кожною людиною керує бажання бути в безпеці. Так само цей постійний пошук безпеки є основоположним для кожного аспекту ІТ сьогодні. Великі обсяги даних, які розумно відфільтровані та чітко оцінені, залишаються ключовими. Питання в тому, як здійснити цей подвиг.

Аналітика безпеки — це проактивний підхід до кібербезпеки, який використовує можливості збору, агрегації та аналізу даних для виконання життєво важливих функцій безпеки, які виявляють, аналізують та пом'якшують кіберзагрози. Інструменти аналітики безпеки, такі як виявлення загроз і моніторинг безпеки, розгортаються з метою виявлення та розслідування інцидентів безпеки або потенційних загроз, таких як зовнішні шкідливі програми, цілеспрямовані атаки та зловмисні інсайдери. Завдяки здатності виявляти ці загрози на ранніх етапах, фахівці з безпеки мають можливість зупинити їх до того, як вони проникнуть в мережеву інфраструктуру, скомпрометують цінні дані та активи або іншим чином завдадуть шкоди організації.

Рішення для аналітики безпеки об'єднують дані з численних джерел, які включають дані про поведінку кінцевих точок і користувачів, бізнес-додатки, журнали подій операційної системи, брандмауери, маршрутизатори, антивірусні сканери, зовнішні дані про загрози та контекстні дані, серед

					КПІ.ІТ-8427.045440.02.81	Арк.
Змін.	Арк.	№ докум.	Підп.	Дата.		
						51

іншого[7]. Об'єднання та співвіднесення цих даних дає організаціям один основний набір даних для роботи, що дозволяє фахівцям із безпеки застосовувати відповідні алгоритми та створювати швидкий пошук для виявлення ранніх індикаторів атаки. Крім того, технології машинного навчання також можна використовувати для аналізу загроз і даних майже в реальному часі.

Безпека баз даних — це різні заходи, які вживають організації для забезпечення захисту своїх баз даних від внутрішніх і зовнішніх загроз. Безпека бази даних включає захист самої бази даних, даних, які вона містить, її системи керування базою даних і різних програм, які мають до неї доступ.

Безпека даних є головною проблемою. Новини про нові порушення даних майже щотижня з'являються в заголовках газет, описуючи компроміси, які впливають на тисячі користувачів.

Хороша новина полягає в тому, що MongoDB має все необхідне для забезпечення найкращих практик безпеки, від шифрування до аутентифікації, контролю доступу та аудиту.

Для того аби убезпечити пароль користувачів було використано функцію bcrypt. Функція хешування bcrypt дозволяє нам створити платформу безпеки паролів, яка масштабується з обчислювальною потужністю і завжди хешує кожен пароль із солі.

Код з файлу models/user.js:

```
UserSchema.methods.encryptPassword = function (password) {  
  return bcrypt.hashSync(password, bcrypt.genSaltSync(5), null);  
};
```

Висновки до розділу

В даному розділі проаналізовано процес подачі заявки на відпустку. Змодельована архітектура програми. Представлена стратегія реалізації зберігання даних на пристрої. Було обґрунтовано використання фреймворку

Node Express, а також описано та обґрунтовано використання MongoDB задля забезпечення безпечного зберігання даних. Також розглянуто проблему безпеки у сучасності та представлені діючі реалізації вирішення зберігання паролей користувачів у базі даних.

					КПІ.ІТ-8427.045440.02.81	Арк.
Змін.	Арк.	№ докум.	Підп.	Дата.		53

3 АНАЛІЗ ЯКОСТІ ТА ТЕСТУВАННЯ ПРОГРАМНОГО ЗАБЕЗПЕЧЕННЯ

3.1 Аналіз якості ПЗ

Іноді ціна помилки може бути занадто високою. Історія знає багато прикладів ситуацій, коли недоліки програмного забезпечення призводили до витрат на мільярди доларів або навіть до жертв: від кав'ярень Starbucks, які були змушені роздавати безкоштовні напої через несправність реєстру, до того, що військовий літак F-35 не міг виявити цілі правильно через несправність радара.

Щоб гарантувати, що випущене програмне забезпечення є безпечним і функціонує належним чином, вводиться поняття якості програмного забезпечення. Зазвичай його визначають як «ступінь, до якої задовольняються явні чи неявні вимоги та очікування». Ці так звані явні та неявні очікування відповідають двом основним рівням якості програмного забезпечення:

Функціональний - продукт відповідає функціональним (явним) вимогам і специфікаціям дизайну. Цей аспект фокусується на фактичному використанні програмного забезпечення, з точки зору користувача: його особливості, продуктивність, простота використання, відсутність дефектів.

Нефункціональні - внутрішні характеристики та архітектура системи, тобто структурні (неявні) вимоги. Це включає підтримку коду, чіткість, ефективність та безпеку.

Структурною якістю програмного забезпечення зазвичай важко керувати: воно покладається здебільшого на досвід команди інженерів і може бути впевненим шляхом перегляду коду, аналізу та рефакторингу. У той же час функціональні аспекти можуть бути забезпечені за допомогою спеціального набору заходів з управління якістю, включаючи забезпечення якості, контроль якості та тестування.

Ці три терміни, які часто використовуються як взаємозамінні, стосуються різних аспектів управління якістю програмного забезпечення. Незважаючи на спільну мету забезпечення найкращої якості продукту за структурою та функціями, вони використовують різні підходи для виконання цього завдання, як показано на малюнку 3.1.



Рисунок 3.1 – QA, QC та Тестування в процесі розробки програмного забезпечення

Забезпечення якості — це широкий термін, який пояснюється в блозі тестування Google як «безперервне та послідовне вдосконалення та підтримка процесу, що забезпечує роботу з контролю якості». Як випливає з визначення, QA більше зосереджується на організаційних аспектах управління якістю, контролю за узгодженістю виробничого процесу.

За допомогою контролю якості команда перевіряє, чи відповідає продукт функціональним вимогам. Згідно з Investopedia, це «процес, за допомогою якого підприємства прагнуть зберегти або покращити якість продукції та зменшити чи усунути виробничі помилки». Ця дія застосовується до готового продукту і виконується до випуску продукту. З точки зору

виробництва, це все одно, що знімати випадкові елементи з конвеєрної стрічки, щоб перевірити, чи відповідають вони вимогам.

Тестування – це основна діяльність, спрямована на виявлення та вирішення технічних проблем у вихідному коді програмного забезпечення та оцінку загальної зручності, продуктивності, безпеки та сумісності продукту. Він має дуже вузьку спрямованість і виконується інженерами-тестувальниками паралельно з процесом розробки або на спеціальному етапі тестування (залежно від методології циклу розробки програмного забезпечення). Створення високоякісного веб-додатка вимагає багато тестування, яке, якщо виконувати його вручну, може бути втомливим і тривалим. З цієї причини багато команд QA покладаються на автоматизацію для створення швидких, ефективних та надійних тестових випадків для своїх веб-додатків.

Автоматизація тестування перевантажує ці рутинні й повторювані завдання тестування від людей до машин. Тести порівнюють фактичні результати з прогнозованими. Цей підхід може допомогти знайти помилки в конкретних операціях і простих випадках використання (наприклад, вхід, створення нового облікового запису, скидання пароля).

Автоматизуючи тестування веб-додатків, тестувальники можуть заощадити час і зусилля на одноманітних завданнях. Автоматизовані тести можна виконувати безперервно або планувати з інтервалами. Це розвантажує тестувальника від трудомістких завдань, і вони можуть зосередитися на дослідницькому тестуванні або інших тестах, які потребують людської точки зору.

Автоматизація тестування — це використання програмного забезпечення (окремого від програмного забезпечення, що тестується) для контролю виконання тестів. Це дозволяє програмним роботам виконувати повторювані завдання та імітувати взаємодію кінцевого користувача з

тестованою системою, щоб збільшити діапазон, глибину та надійність своїх зусиль із забезпечення якості.

Тим не менш, автоматизація не є системою plug-and-play, яка не вимагає втручання людини. Ефективна автоматизація вимагає від тестувальників глибокого знання програмного забезпечення, що тестується, а також настрою «автоматизація спочатку».

Існують різні тестові приклади, які ви можете автоматизувати залежно від ваших бізнес-цілей.

Більше компонентів життєвого циклу автоматизовано, але ви повинні вибрати ті, які найкраще відповідають можливостям вашої команди тестування.

Ось типи тестів, які можна автоматизувати для веб-додатків.

Функціональне тестування

Функціональне тестування використовується для того, щоб переконатися, що функціональність програмного забезпечення працює так, як призначено для кінцевого користувача.

Один кінцевий користувач може привести до збою всієї системи за лічені хвилини, навіть після того, як тести пристрою, інтеграції та продуктивності пройшли. Зазвичай це відбувається тому, що користувач робить те, чого розробники не очікували.

Автоматизація тестів гарантує, що веб-додаток працює належним чином. Однак автоматизація функціональних тестів не може повністю замінити людську перспективу. Ось чому ви завжди повинні доповнювати свої автоматизовані тестові випадки дослідницьким тестуванням, щоб забезпечити позитивний досвід користувача.

Регресійне тестування

Регресійне тестування можна описати як «повторне функціональне тестування». Він використовується, щоб переконатися, що функціональність програмного забезпечення продовжує працювати після того, як його частини

були змінені новим кодом або конфігураціями. Наприклад, коли створюються нові функції, регресійне тестування гарантує, що старі функції програмного забезпечення продовжують працювати належним чином.

Кросс-браузерне тестування

Кросс-браузерне тестування гарантує, що ваша веб-програма працює належним чином у різних браузерах, як на комп'ютері, так і на мобільному пристрої.

Браузери, як правило, оновлюються досить часто, а це означає, що до того часу, коли ви будете готові до розгортання своєї програми, вона може не працювати належним чином у ваших цільових браузерах.

Користувачі можуть мати останню версію браузера під час використання старої операційної системи. Автоматизуючи міжбраузерне тестування, ви гарантуєте, що несумісність, подібна щойно згаданій, буде знайдена ще до того, як вона досягне кінцевого користувача.

Тестування продуктивності

Тестування продуктивності, таке як стресове та навантажувальне тестування, гарантує, що веб-додаток витримує тривалі періоди активності або пікове навантаження користувачів. Досягнення необхідних стресових умов або рівня навантаження було б неможливим, якщо б це виконувалося вручну, тому автоматизація є ключем до підтвердження того, що ваша програма може працювати в будь-якій ситуації.

Для автоматизації тестування веб-додатку можна використовувати різні доступні утиліти, наприклад, LoadNinja. LoadNinja — це хмарна платформа для навантажувального тестування та тестування продуктивності для веб-додатків і веб-сервісів. Це допомагає розробникам, командам QA та інженерам з продуктивності перевірити, чи їх веб-сервери витримують велике навантаження та чи є сервери надійними та масштабованими.

LoadNinja допоможе відповісти на такі запитання:

					КПІ.IT-8427.045440.02.81	Арк.
Змін.	Арк.	№ докум.	Підп.	Дата.		
						58

- Скільки користувачів можуть одночасно працювати з моїм веб-додатком без помітного сповільнення?
- Яке навантаження може призвести до збою сервера?
- Як зміниться час відгуку при збільшенні навантаження?
- Як зміни апаратного та програмного забезпечення впливають на продуктивність сервера?
- І так далі.

За допомогою LoadNinja можна створювати як тести на основі інтерфейсу користувача, які імітують дії користувача в тестованому веб-додатку, так і тести API, які працюють подібно до того, як клієнт звертається до сервера через API.

3.2 Опис процесів тестування

Щоб перевірити свій веб-сервер, ви можете створити або тести інтерфейсу користувача, або тести API, або обидва. Дивіться нижче.

1. Створіть тест інтерфейсу користувача або API

Це «однокористувацькі» тести, кожен з яких представляє процедуру використання програми, як-от розміщення замовлення або пошук у каталозі продуктів. Тест UI моделює дії одного користувача на тестованому веб-сайті; тест API моделює поведінку однієї клієнтської програми. Потім ви запускаєте ці тести кілька разів, щоб імітувати велике навантаження на тестований веб-додаток.

Ви записуєте тести інтерфейсу користувача у вбудованому браузері LoadNinja. Просто введіть URL-адресу та почніть запис!

Що стосується тестів API, то ви створюєте їх вручну, один запит до іншого.

Ви можете додати перевірки як до UI, так і до тестів API, щоб перевірити вміст відповіді та час відповіді сервера.

2. Створіть сценарій навантажувального тестування

Сценарій – це визначення вашого навантажувального тесту. Сценарій поєднує один або кілька «однокористувацьких» тестів, визначає кількість віртуальних користувачів (VU), параметри наростаючого завантаження та інші деталі.

3. Запустіть сценарій

LoadNinja виконує навантажувальні тести в хмарі та автоматично надає хмарні машини на час тестування. Щоб запустити тест інтерфейсу користувача, кожен віртуальний користувач використовує реальний браузер для відтворення тестових дій, тому вони взаємодіють з вашим веб-додатком так само, як це роблять реальні користувачі, включаючи час на роздуми та запуск сценаріїв на стороні клієнта.

Під час запуску LoadNinja відображає діаграми в режимі реального часу, щоб допомогти вам відстежувати хід тестування та стан вашої програми.

4. Проаналізуйте результати

Після завершення тестування LoadNinja створює звіт, що містить показники продуктивності, зібрані під час тестового запуску. Ці дані допомагають оцінити продуктивність вашого сервера та перевірити, чи відповідає він умовам SLA.

Використовувати LoadNinja необхідно задля досягнення таких можливостей тестування:

- Створювання реальних навантажувальних тестів, які моделюють реальні дії користувача. За допомогою LoadNinja ви записуєте дії користувачів на перевіреному веб-сайті, а потім моделюєте записані транзакції за допомогою сотень або тисяч віртуальних користувачів. Таким чином, кожен віртуальний користувач діє дуже подібно до реального користувача, тобто ваші навантажувальні тести близькі до реального життя, вони не просто запитують одну URL-адресу веб-сайту. Це відрізняється від того, що пропонують деякі інші рішення для онлайн-тестування.

– Використовуйте реальні браузери для моделювання реального користувача. LoadNinja відтворює записаний трафік у реальних браузерах. Кожен віртуальний користувач працює у своєму браузері, вони не впливають один на одного. LoadNinja точно моделює дії користувача, включаючи паузи між діями. Якщо деякі перевірені сторінки запускають код сценарію на стороні клієнта, цей код також виконується під час тестового запуску в браузері. Тобто ваш навантажувальний тест працює так, як працюють сотні й тисячі реальних користувачів. Це суперечить багатьом конкурентним онлайн-рішенням і рішенням для настільних комп'ютерів, які записують і відтворюють HTTP-запити і не імітують реальні дії користувача та реальне функціонування браузера.

– Створюйте незалежні від платформи навантажувальні тести. LoadNinja не залежить від платформи. Ви можете створювати та запускати тести в браузері на будь-якому комп'ютері, який має доступ до Інтернету.

– Створюйте тести швидко та легко. Ви створюєте тести інтерфейсу користувача, записуючи дії користувача у вбудованому браузері, а тести API створюєте у вбудованому редакторі запитів. Не потрібно кодувати, завантажувати або встановлювати плагіни для браузера.

– Скористайтесь перевагами чітких дій користувача в тестах. LoadNinja записує дії користувача у вигляді зрозумілих тестових команд, а не набору важко читаних запитів і відповідей. Вам не потрібно витрачати свій час і зусилля на налаштування записаних сценаріїв, щоб підтримувати стан сеансу або надавати дані для користувача. Тобто кореляція даних не потрібна.

– За потреби масштабуйте свої навантажувальні тести. Для моделювання поведінки користувача в реальному житті потрібні ресурси. У випадку настільних рішень вам знадобляться додаткові машини для запуску агентів навантаження. LoadNinja працює в хмарі, тому вам не потрібно шукати додаткові комп'ютери-«агенти», вам не потрібно шукати місце на диску для збереження записаних скриптів і величезної кількості накопичених

результатів тестування. LoadNinja робить все це автоматично за вас. Все, що вам потрібно зробити, це вибрати потрібну кількість віртуальних користувачів для моделювання.

– Записуйте тести на популярних дозволах екрана. Сучасні веб-додатки виглядають дещо по-різному на різних пристроях і роздільних можливостях екрана. За допомогою LoadNinja ви можете легко створювати навантажувальні тести, які будуть імітувати користувачів, які працюють з найпопулярнішими роздільними можливостями екрана для ноутбуків, iPad і мобільних телефонів.

– Спостерігайте за активністю віртуальних користувачів у режимі реального часу. LoadNinja забезпечує підключення до браузерів, які імітують дії користувача, тому ви можете легко бачити вікно браузера та стежити за тим, як працює той чи інший віртуальний користувач.

– Заощаджуйте на об'єкті для тестування. LoadNinja розміщено на Amazon Web Services, тому вам не потрібен власний сервер для цілей навантажувального тестування.

LoadNinja забезпечує конкретний опис отриманих результатів та відображує звіт з отриманого результату. Приклад вказано на рисунках 3.1 – 3.5.

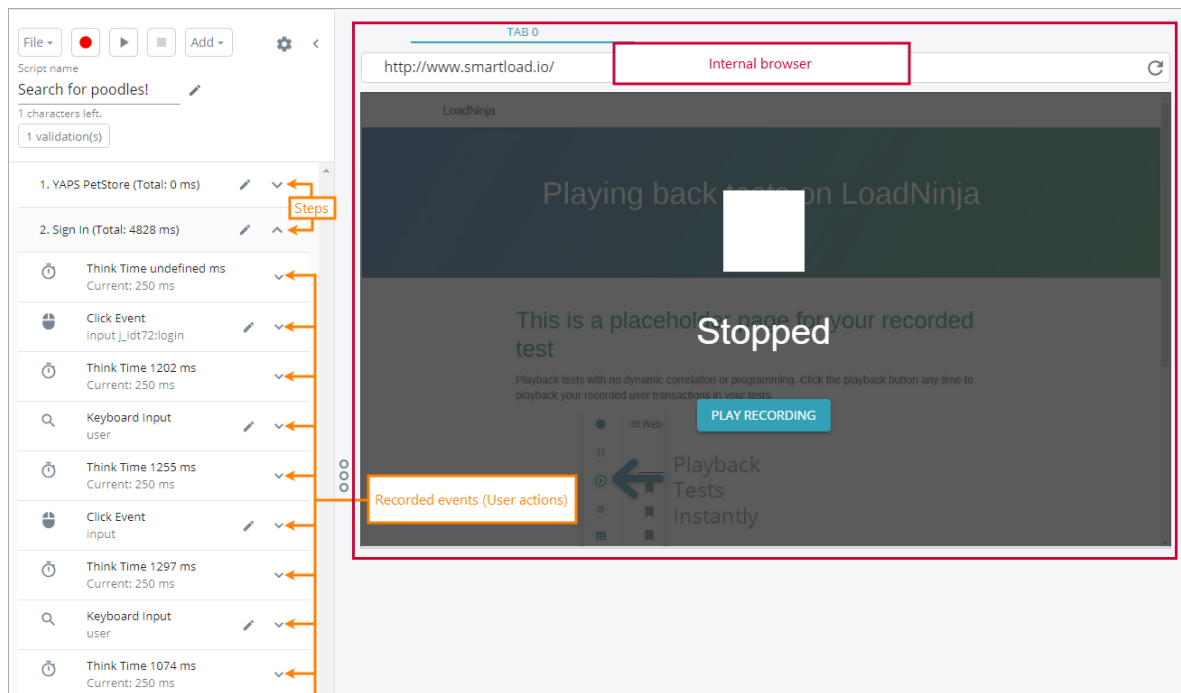


Рисунок 3.1 – LoadNinja записує всі дії.

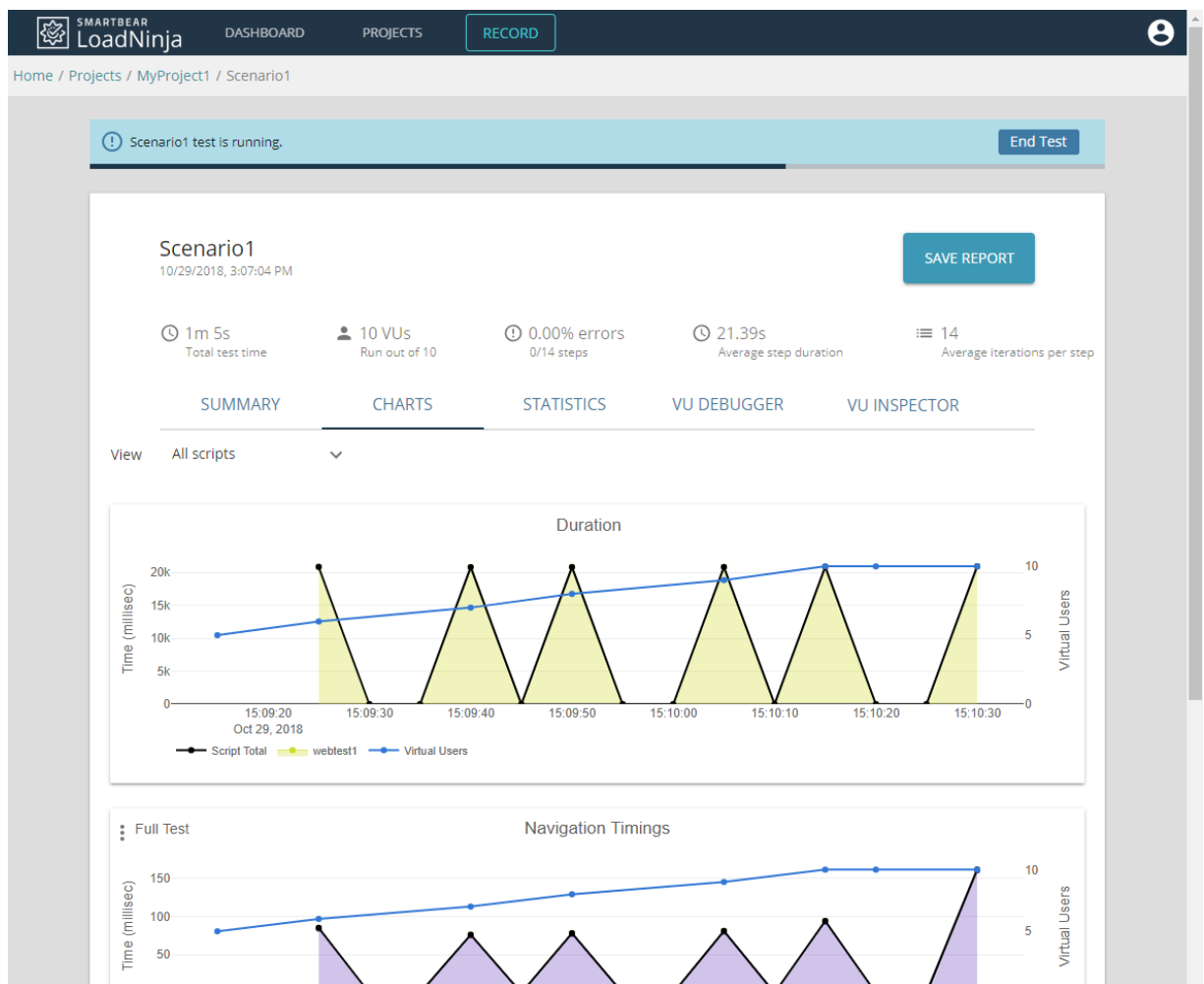


Рисунок 3.2 – На вкладці «Діаграми» накопичені результати у графічній формі.

SMARTBEAR LoadNinja

DASHBOARD
PROJECTS
RECORD

Home / Projects / MyProject1 / Scenario1

Scenario1 test is running.

End Test

Scenario1

10/29/2018, 3:07:04 PM

SAVE REPORT

50s
Total test time

10 VUs
Run out of 10

0.00% errors
0/12 steps

21.39s
Average step duration

12
Average iterations per step

SUMMARY

CHARTS

STATISTICS

VU DEBUGGER

VU INSPECTOR

webtest1 : User 1

http://petstore.smartload.io/applicationPetstore/sh...

webtest1 : User 2

http://petstore.smartload.io/applicationPetstore/sh...

webtest1 : User 3

http://petstore.smartload.io/applicationPetstore/sh...

webtest1 : User 4

http://petstore.smartload.io/applicationPetstore/sh...

Рисунок 3.3 – На вкладці VU Inspector робочі столи віддалених хмарних машин, на яких працюють віртуальні користувачі.

AnotherScenario

10/26/2018, 5:22:27 PM

SAVE REPORT

35s
Total test time

8 VUs
Run out of 8

6.25% errors
4/64 steps

2.00s
Average step duration

64
Average iterations per step

SUMMARY

CHARTS

STATISTICS

VU DEBUGGER

VU INSPECTOR

Time ↓	Script	URL	Step Name	Address	User	Iteration	Error Code	Error Message
10/26/2018, 5:2...	WebTest7	http://petstore... Products_for_cat...	172.31.4.60	1	1	200	AssertionError: CheckPoint1-(d...	
10/26/2018, 5:2...	WebTest7	http://petstore... Products_for_cat...	172.31.4.60	3	1		AssertionError: CheckPoint1-(d...	
10/26/2018, 5:2...	WebTest7	http://petstore... Products_for_cat...	172.31.4.60	4	1	200	AssertionError: CheckPoint1-(d...	
10/26/2018, 5:2...	WebTest7	http://petstore... Products_for_cat...	172.31.4.60	2	1		AssertionError: CheckPoint1-(d...	

1-4 of 4 < >

Рисунок 3.4 – На вкладці VU Debugger інформація про помилки, які виникають під час виконання.

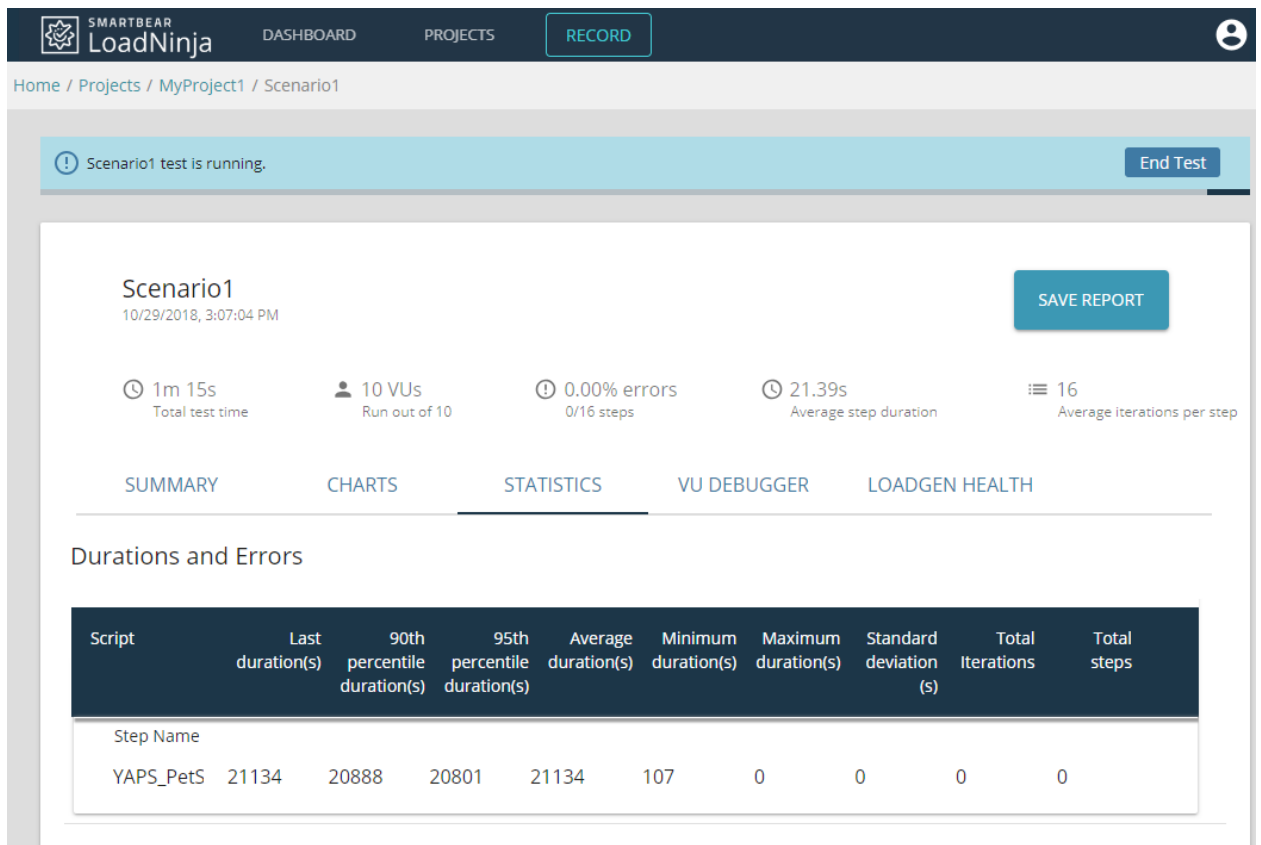


Рисунок 3.5 – На вкладці «Статистика» наведено результати окремих скриптів

3.3 Опис контрольного прикладу

Для детального тестування розробленого веб додатку було обрано 3 модулі основного функціоналу:

- логін у систему;
- запит на відпустку;
- додавання нового співробітника.

Таблиця 3.1 – Успішний логін у систему

Мета тесту	Коректна робота сторінки при логіні користувача з дійсними кредитами входу
Схема виконання	Користувач вводить коректні дані для логіну та натискає на кнопку «Login»
Очікуваний результат	Успішний вхід до системи, перехід на сторінку /home
Дійсний результат	Успішний вхід до системи, перехід на сторінку /home

Таблиця 3.2 – Неуспішний логін у систему

Мета тесту	Коректна робота сторінки при логіні користувача з не дійсними кредитами входу
Схема виконання	Користувач вводить не коректні дані для логіну та натискає на кнопку «Login»
Очікуваний результат	Неуспішний вхід в систему. Вказується повідомлення про помилку в кредитах
Дійсний результат	Неуспішний вхід в систему. Вказується повідомлення про помилку в кредитах

Таблиця 3.3 – Неуспішний логін в систему при вводі коректного email та хибного паролю

Мета тесту	Перевірка роботи системи при умовах вводу коректного email, але з хибним паролем
Схема виконання	Користувач вводить коректний email користувача та хибний пароль. Натискає на кнопку «Login»
Очікуваний результат	Неуспішний вхід в систему. Вказується повідомлення про помилку в кредитах
Дійсний результат	Неуспішний вхід в систему. Вказується повідомлення про помилку у кредитах

Таблиця 3.4 – Коректна робота кнопки вибору дії для запиту на відпустку

Мета тесту	Коректна робота кнопки вибору дії для запиту на відпустку
Схема виконання	Користувач натискає на кнопку “Leave”
Очікуваний результат	Перехід на сторінку /add-vacations
Дійсний результат	Перехід на сторінку /add-vacations

Таблиця 3.5 – Успішна відправка запиту на відпустку

Мета тесту	Перевірка роботи системи при вводі коректних даних при запиті на відпустку
Схема виконання	Користувач вводить дані про дати, кількість днів, тип відпустки та натискає кнопку «Зберегти»
Очікуваний результат	Успішне збереження заявки та сповіщення про це.
Дійсний результат	Успішне збереження заявки та сповіщення про це.

Таблиця 3.6 – Неуспішна відправка запиту на відпустку

Мета тесту	Перевірка роботи системи при вводі не коректних даних при запиті на відпустку
Схема виконання	Користувач вводить не коректні дані про дати, кількість днів, тип відпустки та натискає кнопку «Зберегти»
Очікуваний результат	Заявка не збережена, отримано сповіщення про це
Дійсний результат	Заявка не збережена, отримано сповіщення про це

Таблиця 3.7 – Введення буквених даних у поле для вводу дат при запиті на відпустку

Мета тесту	Перевірка роботи системи при введенні буквених даних у поле для введення дат при запиті на відпустку
Схема виконання	Користувач намагається ввести буквені дані у поле для дат
Очікуваний результат	Система не дає можливості це зробити
Дійсний результат	Система не дає можливості це зробити

Таблиця 3.8 - Коректна робота кнопки вибору дії для додавання нового співробітника

Мета тесту	Коректна робота кнопки вибору дії для додавання нового співробітника
Схема виконання	Користувач натискає на кнопку “Add Employee”
Очікуваний результат	Перехід на сторінку /add-employee
Дійсний результат	Перехід на сторінку /add-employee

Таблиця 3.9 – Успішне додавання нового співробітника

Мета тесту	Перевірка роботи системи при вводі коректних даних про нового співробітника
Схема виконання	Адміністратор системи вводить на сторінці дані про співробітника та натискає кнопку «Додати»
Очікуваний результат	Нового співробітника додано отримано сповіщення про це
Дійсний результат	Нового співробітника додано отримано сповіщення про це

Таблиця 3.10 - Неуспішне додавання нового співробітника

Мета тесту	Перевірка роботи системи при вводі не коректних даних про нового співробітника
Схема виконання	Адміністратор системи вводить на сторінці не коректні дані про співробітника та натискає кнопку «Додати»
Очікуваний результат	Нового співробітника не додано, отримано сповіщення про це
Дійсний результат	Нового співробітника не додано, отримано сповіщення про це

Таблиця 3.11 – Введення у поле для телефону номер не підходящий масці вводу

Мета тесту	Перевірити роботу маски вводу на полі для вводу номеру телефону
Схема виконання	Введення у поле вводу телефону номер не підходящий масці
Очікуваний результат	Отримане сповіщення про не коректний ввід даних
Дійсний результат	Отримане сповіщення про не коректний ввід даних

Таблиця 3.12 – Перевірка виходу з системи

Мета тесту	Перевірка виходу із системи при натисненні кнопки “Log out”
Схема виконання	Користувач натискає кнопку “Log out”
Очікуваний результат	Система повертається до сторінки /login
Дійсний результат	Система повертається до сторінки /login

Висновки до розділу

В даному розділі проведено тестування розробленого програмного забезпечення за допомогою стандарту SLA. Для цього було виконано тестування за допомогою LoadNinja. Також виконано опис функцій які необхідно було протестувати, сформовані вимоги. За результатом виконання описаних тестів було визначено, що усі тести пройдені успішно, система працює справно.

					КПІ.ІТ-8427.045440.02.81	Арк.
Змін.	Арк.	№ докум.	Підп.	Дата.		72

4 ВПРОВАДЖЕННЯ ТА СУПРОВІД ПРОГРАМНОГО ЗАБЕЗПЕЧЕННЯ

4.1 Розгортання програмного забезпечення

Зазвичай багато хто визначає розгортання програмного забезпечення як процес розгортання, налаштування та оновлення програмних додатків, щоб забезпечити максимальну оптимізацію, безпеку та сумісність в ІТ-середовищі.

Це розгортання може відбуватися вручну або автоматично, у той час, коли працівники організації вважають найменш нав'язливими. Для великих розгортань це може призвести до поетапних випусків, щоб звести до мінімуму перерви в продуктивності співробітників і зменшити навантаження на систему, включаючи підтримку після розгортання.

Для розгортання веб-додатку необхідно завантажити усі файли програми на комп'ютер.

Необхідно перевірити чи встановлено NodeJS на пристрій. Якщо ні, то перейти на офіційний сайт та встановити останню версію за інструкцією, наведеною на офіційному сайті.

Також необхідно встановити MongoDB сервер для забезпечення роботи з базою даних.

- 1) Помістіть усі ці папки та файли в одну папку.
- 2) Відкрийте цю папку в будь-якому IDE.
- 3) Запустіть сервер bin/www, натиснувши зелену кнопку зі стрілкою в правому куті вікна.
- 4) Тепер сервер запущено.
- 5) Відкрийте будь-який браузер.
- 6) В адресному рядку напишіть localhost:3000, де 3000 — порт, який використовує ця програма.
- 7) Браузер перенаправить на домашню сторінку програми.
- 8) Програма запущена.

4.2 Робота з програмним забезпеченням

Після успішного імпортування всіх файлів програми та запуску серверу bin/www для користувача є доступним усі модулі функціоналу програми.

Сервіс працює за адресою localhost:3000, при введенні цієї адреси у адресну строку браузера можна перейти на сторінку логіну у систему. Детальна робота з системою описана у керівництві користувача.

Висновки до розділу

У даному розділі описано перелік необхідних встановлених програм задля успішного запуску розробленого програмного забезпечення. Також створено керівництво користувача, де викладені детальні інструкції роботи із системою. Було детально описано, що необхідно виконати для розгортання програми.

					КПІ.IT-8427.045440.02.81	Арк. 74
Змін.	Арк.	№ докум.	Підп.	Дата.		

ВИСНОВКИ

В результаті виконання дипломного проєкту було спроектовано та розроблено програмне забезпечення HR-порталу, який спрощує та автоматизує основну частину роботи HR-менеджера. Тому мету розробити та зібрати в одному місці інструменти для вирішення задач керування персоналом згідно основних задач HR можна вважати досягнутою.

Реалізовано наступні програмні функції.

- Окремі облікові записи користувачів для всіх адміністраторів і співробітників.
- Окремі погляди на дані та чіткий доступ до даних для членів на основі їхніх привілеїв.
- Реалізація можливості адміністрування відпустками співробітників.
- Реєстрація співробітника/адміністратора.
- Контроль відвідуваності всіх співробітників.
- Управління заробітною платою всіх працівників.
- Ведення обліку освітнього та виробничого стажу працівника.
- Управління поточними проектами, призначеними співробітникам, всередині організації.

В якості середовища розробки обрано NodeJS.

У якості БД використано MongoDB.

Розроблене програмне забезпечення протестовано на пристроях з різними версіями браузеру і різними розмірами екрана, щоб переконатися, що програма відображатиметься акуратно на різних пристроях.

СПИСОК ВИКОРИСТАНИХ ДЖЕРЕЛ

- 1) Farnham D., Pimlott J., Farnham D. Understanding Industrial Relations. 4-те вид. Cassell Academic, 1990. 244 с.
- 2) Armstrong M. A handbook of personnel management practice. 6-те вид. London : Kogan Page, 1999. 959 с.
- 3) Kleppe A. G. MDA explained: The model driven architecture : practice and promise. Boston : Addison-Wesley, 2003. 170 с.
- 4) Rumbaugh J. The unified modeling language reference manual. Reading, Mass : Addison-Wesley, 1999. 550 с.
- 5) Dessler G. Human Resource Management, Plus MyManagementLab with Pearson EText. Pearson Education, Limited, 2013. 494 с.
- 6) Dessler G. Personnel management: Modern concepts & techniques. 2nd ed. Reston, Va : Reston Pub. Co., 1981. 557 p.
- 7) Casciaro M. Node.js Design Patterns: Master a series of patterns and techniques to create modular, scalable, and efficient applications. Packt Publishing, 2014. 454 с.
- 8) Hoque S. Full-Stack React Projects: Modern web development using React 16, Node, Express, and MongoDB. Packt Publishing, 2018. 470 p.
- 9) Lychkan L. P. Personnel management: personnel policy and personnel planning. Uchenyy Sovet (Academic Council). 2021. № 5. С. 339–351. URL: <https://doi.org/10.33920/nik-02-2105-02> (дата звернення: 10.06.2022).
- 10) Human Resource Management. Human Resource Management. 2014. Т. 53, № 6. С. no. URL: <https://doi.org/10.1002/hrm.21696> (дата звернення: 10.06.2022).

ДОДАТКИ

					КПІ.ІТ-8427.045440.02.81	Арк.
Змін.	Арк.	№ докум.	Підп.	Дата.		77

Факультет інформатики та обчислювальної техніки
Кафедра інформатики та програмної інженерії

“ЗАТВЕРДЖЕНО”

Завідувач кафедри

_____ Едуард ЖАРІКОВ

“ ____ ” _____ 2022 р.

**ПОРТАЛ ОСОБИСТОГО КАБІНЕТУ СПІВРОБІТНИКА СУЧАСНОЇ
КОМПАНІЇ**

Опис програми

КПІ.ІТ-8427.045440.03.13

“ПОГОДЖЕНО”

Керівник проєкту:

_____ Тетяна ЛІХОУЗОВА

Нормоконтроль:

_____ Катерина ЛІЩУК

Виконавець:

_____ Світлана ТРОФІМОВА

Київ – 2022

ТЕКСТ ПРОГРАМНОГО КОДУ

Тексти програмного коду

Портал особистого кабінету співробітника сучасної компанії

(Найменування програми (документа))

CD-R

(Вид носія даних)

27 арк, 11700 Кб

(Обсяг програми, арк., Кб)

Київ – 2022

					КПІ.ІТ-8427.045440.03.13	Арк. 79
Змін.	Арк.	№ докум.	Підп.	Дата.		

Файл bin/www.js

```
var app = require('../app');
var debug = require('debug')('hrms:server');
var http = require('http');
var port = normalizePort(process.env.PORT || '3000');
app.set('port', port);

var server = http.createServer(app);

server.listen(port);
server.on('error', onError);
server.on('listening', onListening);

function normalizePort(val) {
  var port = parseInt(val, 10);

  if (isNaN(port)) {
    return val;
  }

  if (port >= 0) {
    return port;
  }

  return false;
}

function onError(error) {
  if (error.syscall !== 'listen') {
    throw error;
  }

  var bind = typeof port === 'string'
    ? 'Pipe ' + port
    : 'Port ' + port;

  switch (error.code) {
    case 'EACCES':
      console.error(bind + ' requires elevated privileges');
      process.exit(1);
      break;
    case 'EADDRINUSE':
      console.error(bind + ' is already in use');
      process.exit(1);
      break;
    default:
      throw error;
  }
}

function onListening() {
  var addr = server.address();
  var bind = typeof addr === 'string'
    ? 'pipe ' + addr
    : 'port ' + addr.port;
  debug('Listening on ' + bind);
}
```

Файл routes\admin.js

```
var express = require('express');
var router = express.Router();
var passport = require('passport');
var User = require('../models/user');
var Project = require('../models/project');
var csrf = require('csrf');
var csrfProtection = csrf();
var config_passport = require('../config/passport.js');
var moment = require('moment');
var Leave = require('../models/leave');
var Attendance = require('../models/attendance');

router.use('/', isLoggedIn, function isAuthenticated(req, res, next) {
    next();
});

router.get('/', function viewHome(req, res, next) {
    res.render('Admin/adminHome', {
        title: 'Admin Home',
        csrfToken: req.csrfToken(),
        userName: req.session.user.name
    });
});

router.get('/view-profile', function viewProfile(req, res, next) {
    User.findById(req.session.user._id, function getUser(err, user) {
        if (err) {
            console.log(err);
        }
        res.render('Admin/viewProfile', {
            title: 'Profile',
            csrfToken: req.csrfToken(),
            employee: user,
            moment: moment,
            userName: req.session.user.name
        });
    });
});

router.get('/view-all-employees', function viewAllEmployees(req, res,
next) {

    var userChunks = [];
    var chunkSize = 3;
    //find is asynchronous function
    User.find({$or: [{type: 'employee'}, {type: 'project_manager'}, {type:
'accounts_manager'}]}).sort({_id: -1}).exec(function getUsers(err, docs) {
        for (var i = 0; i < docs.length; i++) {
            userChunks.push(docs[i]);
        }
        res.render('Admin/viewAllEmployee', {
            title: 'All Employees',
            csrfToken: req.csrfToken(),
```

КПІ.IT-8427.045440.03.13					Арк.
Змін.	Арк.	№ докум.	Підп.	Дата.	81

```

        users: userChunks,
        userName: req.session.user.name
    });
});

});

router.get('/add-employee', function addEmployee(req, res, next) {
    var messages = req.flash('error');
    var newUser = new User();

    res.render('Admin/addEmployee', {
        title: 'Add Employee',
        csrfToken: req.csrfToken(),
        user: config_passport.User,
        messages: messages,
        hasErrors: messages.length > 0,
        userName: req.session.user.name
    });

});

router.get('/all-employee-projects/:id', function
getAllEmployeePojects(req, res, next) {
    var employeeId = req.params.id;
    var projectChunks = [];

    //find is asynchronous function
    Project.find({employeeID: employeeId}).sort({_id: -1}).exec(function
findProjectOfEmployee(err, docs) {
        var hasProject = 0;
        if (docs.length > 0) {
            hasProject = 1;
        }
        for (var i = 0; i < docs.length; i++) {
            projectChunks.push(docs[i]);
        }
        User.findById(employeeId, function getUser(err, user) {
            if (err) {
                console.log(err);
            }
            res.render('Admin/employeeAllProjects', {
                title: 'List Of Employee Projects',
                hasProject: hasProject,
                projects: projectChunks,
                csrfToken: req.csrfToken(),
                user: user,
                userName: req.session.user.name
            });
        });
    });

});

});

router.get('/leave-applications', function getLeaveApplications(req, res,
next) {

```



```

var leaveChunks = [];
var employeeChunks = [];
var temp;
//find is asynchronous function
Leave.find({}).sort({_id: -1}).exec(function findAllLeaves(err, docs)
{
    var hasLeave = 0;
    if (docs.length > 0) {
        hasLeave = 1;
    }
    for (var i = 0; i < docs.length; i++) {
        leaveChunks.push(docs[i])
    }
    for (var i = 0; i < leaveChunks.length; i++) {

        User.findById(leaveChunks[i].applicantID, function
getUser(err, user) {
            if (err) {
                console.log(err);
            }
            employeeChunks.push(user);

        })
    }

    // call the rest of the code and have it execute after 3 seconds
    setTimeout(render_view, 900);
    function render_view() {
        res.render('Admin/allApplications', {
            title: 'List Of Leave Applications',
            csrfToken: req.csrfToken(),
            hasLeave: hasLeave,
            leaves: leaveChunks,
            employees: employeeChunks, moment: moment, userName:
req.session.user.name
        });
    }
});

});

router.get('/respond-application/:leave_id/:employee_id', function
respondApplication(req, res, next) {
    var leaveID = req.params.leave_id;
    var employeeID = req.params.employee_id;
    Leave.findById(leaveID, function getLeave(err, leave) {

        if (err) {
            console.log(err);
        }
        User.findById(employeeID, function getUser(err, user) {
            if (err) {
                console.log(err);
            }
            res.render('Admin/applicationResponse', {
                title: 'Respond Leave Application',
                csrfToken: req.csrfToken(),
                leave: leave,

```

```

        employee: user,
        moment: moment, userName: req.session.user.name
    });

    })

});

});

router.get('/employee-profile/:id', function getEmployeeProfile(req, res,
next) {
    var employeeId = req.params.id;
    User.findById(employeeId, function getUser(err, user) {
        if (err) {
            console.log(err);
        }
        res.render('Admin/employeeProfile', {
            title: 'Employee Profile',
            employee: user,
            csrfToken: req.csrfToken(),
            moment: moment,
            userName: req.session.user.name
        });
    });
});

router.get('/edit-employee/:id', function editEmployee(req, res, next) {
    var employeeId = req.params.id;
    User.findById(employeeId, function getUser(err, user) {
        if (err) {
            res.redirect('/admin/');
        }
        res.render('Admin/editEmployee', {
            title: 'Edit Employee',
            csrfToken: req.csrfToken(),
            employee: user,
            moment: moment,
            message: '',
            userName: req.session.user.name
        });
    });
});

router.get('/edit-employee-project/:id', function editEmployeeProject(req,
res, next) {
    var projectId = req.params.id;
    Project.findById(projectId, function getProject(err, project) {
        if (err) {
            console.log(err);
        }
    }

```

```

        res.render('Admin/editProject', {
            title: 'Edit Employee',
            csrfToken: req.csrfToken(),
            project: project,
            moment: moment,
            message: '',
            userName: req.session.user.name
        });

    });

});

router.get('/add-employee-project/:id', function addEmployeeProject(req,
res, next) {

    var employeeId = req.params.id;
    User.findById(employeeId, function getUser(err, user) {
        if (err) {
            res.redirect('/admin/');
        }
        res.render('Admin/addProject', {
            title: 'Add Employee Project',
            csrfToken: req.csrfToken(),
            employee: user,
            moment: moment,
            message: '',
            userName: req.session.user.name
        });
    });

});

});

router.get('/employee-project-info/:id', function
viewEmployeeProjectInfo(req, res, next) {
    var projectId = req.params.id;
    Project.findById(projectId, function getProject(err, project) {
        if (err) {
            console.log(err);
        }
        User.findById(project.employeeID, function getUser(err, user) {
            if (err) {
                console.log(err);
            }
            res.render('Admin/projectInfo', {
                title: 'Employee Project Information',
                project: project,
                employee: user,
                moment: moment,
                message: '',
                userName: req.session.user.name,
                csrfToken: req.csrfToken()
            });
        });
    });

});

});

```

```

});

router.get('/redirect-employee-profile', function viewEmployeeProfile(req,
res, next) {
    var employeeId = req.user.id;
    User.findById(employeeId, function getUser(err, user) {
        if (err) {
            console.log(err);
        }
        res.redirect('/admin/employee-profile/' + employeeId);
    });
});

});

router.post('/view-attendance', function viewAttendance(req, res, next) {
    var attendanceChunks = [];
    Attendance.find({
        employeeID: req.session.user._id,
        month: req.body.month,
        year: req.body.year
    }).sort({_id: -1}).exec(function viewAttendanceSheet(err, docs) {
        var found = 0;
        if (docs.length > 0) {
            found = 1;
        }
        for (var i = 0; i < docs.length; i++) {
            attendanceChunks.push(docs[i]);
        }
        res.render('Admin/viewAttendanceSheet', {
            title: 'Attendance Sheet',
            month: req.body.month,
            csrfToken: req.csrfToken(),
            found: found,
            attendance: attendanceChunks,
            userName: req.session.user.name,
            moment: moment
        });
    });
});

});

router.get('/view-attendance-current', function
viewCurrentlyMarkedAttendance(req, res, next) {
    var attendanceChunks = [];

    Attendance.find({
        employeeID: req.session.user._id,
        month: new Date().getMonth() + 1,
        year: new Date().getFullYear()
    }).sort({_id: -1}).exec(function getAttendanceSheet(err, docs) {
        var found = 0;
        if (docs.length > 0) {
            found = 1;
        }
        for (var i = 0; i < docs.length; i++) {

```

```

        attendanceChunks.push(docs[i]);
    }
    res.render('Admin/viewAttendanceSheet', {
        title: 'Attendance Sheet',
        month: new Date().getMonth() + 1,
        csrfToken: req.csrfToken(),
        found: found,
        attendance: attendanceChunks,
        moment: moment,
        userName: req.session.user.name
    });
});

});

router.get('/view-employee-attendance/:id', function
viewEmployeeAttendance(req, res, next) {
    var attendanceChunks = [];
    Attendance.find({employeeID: req.params.id}).sort({_id: -
1}).exec(function getAttendanceSheet(err, docs) {
        var found = 0;
        if (docs.length > 0) {
            found = 1;
        }
        for (var i = 0; i < docs.length; i++) {
            attendanceChunks.push(docs[i]);
        }

        User.findById(req.params.id, function getUser(err, user) {

            res.render('Admin/employeeAttendanceSheet', {
                title: 'Employee Attendance Sheet',
                month: req.body.month,
                csrfToken: req.csrfToken(),
                found: found,
                attendance: attendanceChunks,
                moment: moment,
                userName: req.session.user.name
            },
            'employee_name': user.name
        ));
    });
});

});

router.post('/add-employee', passport.authenticate('local.add-employee', {
    successRedirect: '/admin/redirect-employee-profile',
    failureRedirect: '/admin/add-employee',
    failureFlash: true,
}));

router.post('/respond-application', function respondApplication(req, res) {

    Leave.findById(req.body.leave_id, function getLeave(err, leave) {

```

```

        leave.adminResponse = req.body.status;
        leave.save(function saveLeave(err) {
            if (err) {
                console.log(err);
            }
            res.redirect('/admin/leave-applications');
        })
    })

});

router.post('/edit-employee/:id', function editEmployee(req, res) {
    var employeeId = req.params.id;
    var newUser = new User();
    newUser.email = req.body.email;
    if (req.body.designation == "Accounts Manager") {
        newUser.type = "accounts_manager";
    }
    else if (req.body.designation == "Project Manager") {
        newUser.type = "project_manager";
    }
    else {
        newUser.type = "employee";
    }
    newUser.name = req.body.name,
    newUser.dateOfBirth = new Date(req.body.DOB),
    newUser.contactNumber = req.body.number,
    newUser.department = req.body.department;
    newUser.Skills = req.body['skills[]'];
    newUser.designation = req.body.designation;

    User.findById(employeeId, function getUser(err, user) {
        if (err) {
            res.redirect('/admin/');
        }
        if (user.email != req.body.email) {
            User.findOne({'email': req.body.email}, function getUser(err,
user) {
                if (err) {
                    res.redirect('/admin/');
                }
                if (user) {
                    res.render('Admin/editEmployee', {
                        title: 'Edit Employee',
                        csrfToken: req.csrfToken(),
                        employee: newUser,
                        moment: moment,
                        message: 'Email is already in use', userName:
req.session.user.name
                    });
                }
            });
        }
    });

    user.email = req.body.email;
    if (req.body.designation == "Accounts Manager") {
        user.type = "accounts_manager";
    }

```

КПІ.IT-8427.045440.03.13					Арк.
Змін.	Арк.	№ докум.	Підп.	Дата.	88

```

    }
    else if (req.body.designation == "Project Manager") {
        user.type = "project_manager";
    }
    else {
        user.type = "employee";
    }
    user.name = req.body.name,
    user.dateOfBirth = new Date(req.body.DOB),
    user.contactNumber = req.body.number,
    user.department = req.body.department;
    user.Skills = req.body['skills[]'];
    user.designation = req.body.designation;

    user.save(function saveUser(err) {
        if (err) {
            console.log(error);
        }
        res.redirect('/admin/employee-profile/' + employeeId);
    });
});

});

router.post('/add-employee-project/:id', function addEmployeeProject(req,
res) {
    var newProject = new Project();
    newProject.employeeID = req.params.id;
    newProject.title = req.body.title;
    newProject.type = req.body.type;
    newProject.startDate = new Date(req.body.start_date),
    newProject.endDate = new Date(req.body.end_date),
    newProject.description = req.body.description,
    newProject.status = req.body.status;

    newProject.save(function saveProject(err) {
        if (err) {
            console.log(err);
        }
        res.redirect('/admin/employee-project-info/' + newProject._id);
    });
});

});

router.post('/edit-employee-project/:id', function
editEmployeeProject(req, res) {
    var projectId = req.params.id;
    var newProject = new Project();

    Project.findById(projectId, function (err, project) {
        if (err) {
            console.log(err);
        }
        project.title = req.body.title;
        project.type = req.body.type;

```

```

        project.startDate = new Date(req.body.start_date),
        project.endDate = new Date(req.body.end_date),
        project.description = req.body.description,
        project.status = req.body.status;

        project.save(function saveProject(err) {
            if (err) {
                console.log(err);
            }
            res.redirect('/admin/employee-project-info/' + projectId);
        });
    });
});

router.post('/delete-employee/:id', function deleteEmployee(req, res) {
    var id = req.params.id;
    User.findByIdAndRemove({_id: id}, function deleteUser(err) {
        if (err) {
            console.log('unable to delete employee');
        }
        else {
            res.redirect('/admin/view-all-employees');
        }
    });
});

router.post('/mark-attendance', function markAttendance(req, res, next) {
    Attendance.find({
        employeeID: req.session.user._id,
        date: new Date().getDate(),
        month: new Date().getMonth() + 1,
        year: new Date().getFullYear()
    }, function getAttendance(err, docs) {
        var found = 0;
        if (docs.length > 0) {
            found = 1;
        }
        else {

            var newAttendance = new Attendance();
            newAttendance.employeeID = req.session.user._id;
            newAttendance.year = new Date().getFullYear();
            newAttendance.month = new Date().getMonth() + 1;
            newAttendance.date = new Date().getDate();
            newAttendance.present = 1;
            newAttendance.save(function saveAttendance(err) {
                if (err) {
                    console.log(err);
                }
            });
        }
        res.redirect('/admin/view-attendance-current');
    });
});

```

КПІ.IT-8427.045440.03.13					Арк.
Змін.	Арк.	№ докум.	Підп.	Дата.	90


```

    });
    module.exports = router;

    function isLoggedIn(req, res, next) {
        if (req.isAuthenticated()) {
            return next();
        }
        res.redirect('/');
    }

    function notLoggedIn(req, res, next) {
        if (!req.isAuthenticated()) {
            return next();
        }
        res.redirect('/');
    }

    routes/employee.js:
var express = require('express');
var router = express.Router();
var Leave = require('../models/leave');
var Attendance = require('../models/attendance');
var Project = require('../models/project');
var moment = require('moment');
var User = require('../models/user');
var csrf = require('csrf');
var csrfProtection = csrf();
var moment = require('moment');

router.use('/', isLoggedIn, function checkAuthentication(req, res, next) {
    next();
});

router.get('/', function viewHome(req, res, next) {
    res.render('Employee/employeeHome', {
        title: 'Home',
        userName: req.session.user.name,
        csrfToken: req.csrfToken()
    });
});

router.get('/apply-for-leave', function applyForLeave(req, res, next) {
    res.render('Employee/applyForLeave', {
        title: 'Apply for Leave',
        csrfToken: req.csrfToken(),
        userName: req.session.user.name
    });
});

router.get('/applied-leaves', function viewAppliedLeaves(req, res, next) {
    var leaveChunks = [];

    //find is asynchronous function
    Leave.find({applicantID: req.user._id}).sort({_id: -1}).exec(function
getLeaves(err, docs) {
        var hasLeave = 0;

```

```

        if (docs.length > 0) {
            hasLeave = 1;
        }
        for (var i = 0; i < docs.length; i++) {
            leaveChunks.push(docs[i]);
        }

        res.render('Employee/appliedLeaves', {
            title: 'List Of Applied Leaves',
            csrfToken: req.csrfToken(),
            hasLeave: hasLeave,
            leaves: leaveChunks,
            userName: req.session.user.name
        });
    });
});

router.post('/view-attendance', function viewAttendanceSheet(req, res, next) {
    var attendanceChunks = [];
    Attendance.find({
        employeeID: req.session.user._id,
        month: req.body.month,
        year: req.body.year
    }).sort({_id: -1}).exec(function getAttendance(err, docs) {
        var found = 0;
        if (docs.length > 0) {
            found = 1;
        }
        for (var i = 0; i < docs.length; i++) {
            attendanceChunks.push(docs[i]);
        }
        res.render('Employee/viewAttendance', {
            title: 'Attendance Sheet',
            month: req.body.month,
            csrfToken: req.csrfToken(),
            found: found,
            attendance: attendanceChunks,
            moment: moment,
            userName: req.session.user.name
        });
    });
});

});

router.get('/view-attendance-current', function
viewCurrentlyMarkedAttendance(req, res, next) {
    var attendanceChunks = [];

    Attendance.find({
        employeeID: req.session.user._id,
        month: new Date().getMonth() + 1,
        year: new Date().getFullYear()
    }).sort({_id: -1}).exec(function getAttendanceSheet(err, docs) {
        var found = 0;
        if (docs.length > 0) {
            found = 1;
        }
    });
});

```

```

    }
    for (var i = 0; i < docs.length; i++) {
        attendanceChunks.push(docs[i]);
    }
    res.render('Employee/viewAttendance', {
        title: 'Attendance Sheet',
        month: new Date().getMonth() + 1,
        csrfToken: req.csrfToken(),
        found: found,
        attendance: attendanceChunks,
        moment: moment,
        userName: req.session.user.name
    });
});
});

router.get('/view-profile', function viewProfile(req, res, next) {
    User.findById(req.session.user._id, function getUser(err, user) {
        if (err) {
            console.log(err);
        }
        res.render('Employee/viewProfile', {
            title: 'Profile',
            csrfToken: req.csrfToken(),
            employee: user,
            moment: moment,
            userName: req.session.user.name
        });
    });
});

router.get('/view-all-projects', function viewAllProjects(req, res, next) {
    var projectChunks = [];
    Project.find({employeeID: req.session.user._id}).sort({_id: -
1}).exec(function getProjects(err, docs) {
        var hasProject = 0;
        if (docs.length > 0) {
            hasProject = 1;
        }
        for (var i = 0; i < docs.length; i++) {
            projectChunks.push(docs[i]);
        }
        res.render('Employee/viewPersonalProjects', {
            title: 'List Of Projects',
            hasProject: hasProject,
            projects: projectChunks,
            csrfToken: req.csrfToken(),
            userName: req.session.user.name
        });
    });
});
});
});

```

```

router.get('/view-project/:project_id', function viewProject(req, res, next) {

    var projectId = req.params.project_id;
    Project.findById(projectId, function getProject(err, project) {
        if (err) {
            console.log(err);
        }
        res.render('Employee/viewProject', {
            title: 'Project Details',
            project: project,
            csrfToken: req.csrfToken(),
            moment: moment,
            userName: req.session.user.name
        });
    });

});

});

router.post('/apply-for-leave', function applyForLeave(req, res, next) {

    var newLeave = new Leave();
    newLeave.applicantID = req.user._id;
    newLeave.title = req.body.title;
    newLeave.type = req.body.type;
    newLeave.startDate = new Date(req.body.start_date);
    newLeave.endDate = new Date(req.body.end_date);
    newLeave.period = req.body.period;
    newLeave.reason = req.body.reason;
    newLeave.appliedDate = new Date();
    newLeave.adminResponse = 'Pending';
    newLeave.save(function saveLeave(err) {
        if (err) {
            console.log(err);
        }
        res.redirect('/employee/applied-leaves');
    });

});

router.post('/mark-employee-attendance', function markEmployeeAttendance(req,
res, next) {

    Attendance.find({
        employeeID: req.user._id,
        month: new Date().getMonth()+ 1,
        date: new Date().getDate(),
        year: new Date().getFullYear()
    }, function getAttendanceSheet(err, docs) {
        var found = 0;
        if (docs.length > 0) {
            found = 1;
        }
        else {

            var newAttendance = new Attendance();
            newAttendance.employeeID = req.user._id;

```

```

        newAttendance.year = new Date().getFullYear();
        newAttendance.month = new Date().getMonth() + 1;
        newAttendance.date = new Date().getDate();
        newAttendance.present = 1;
        newAttendance.save(function saveAttendance(err) {
            if (err) {
                console.log(err);
            }
        });
    }
    res.redirect('/employee/view-attendance-current');
});

});
module.exports = router;

```

```

function isLoggedIn(req, res, next) {

    if (req.isAuthenticated()) {
        return next();
    }
    res.redirect('/');
}

```

Файл routes/index.js:

```

var express = require('express');
var router = express.Router();
var passport = require('passport');
var User = require('../models/user');
var csrf = require('csrf');
var csrfProtection = csrf();
router.use(csrfProtection);

router.get('/', function viewLoginPage(req, res, next) {
    var messages = req.flash('error');

    res.render('login', {
        title: 'HR-Portal|Log in',
        csrfToken: req.csrfToken(),
        messages: messages,
        hasErrors: messages.length > 0
    });
});

router.get('/logout', isLoggedIn, function logoutUser(req, res, next) {

    req.logout();
    res.redirect('/');
});

router.get('/check-type', function checkTypeOfLoggedInUser(req, res, next) {
    req.session.user = req.user;

```

КПІ.IT-8427.045440.03.13					Арк.
Змін.	Арк.	№ докум.	Підп.	Дата.	95

```

    if (req.user.type == "project_manager") {
        res.redirect('/manager/');
    }
    else if (req.user.type == "accounts_manager") {
        res.redirect('/manager/');
    }
    else if (req.user.type == "employee") {
        res.redirect('/employee/');
    }
    else {
        res.redirect('/admin/');
    }
});

router.post('/login', passport.authenticate('local.signin', {
    successRedirect: '/check-type',
    failureRedirect: '/',
    failureFlash: true

}));

module.exports = router;

function isLoggedIn(req, res, next) {

    if (req.isAuthenticated()) {
        return next();
    }
    res.redirect('/');
}

```

Файл routes/manager.js:

```

var express = require('express');
var router = express.Router();
var User = require('../models/user');
var UserSalary = require('../models/user_salary');
var PaySlip = require('../models/payslip');
var Leave = require('../models/leave');
var Attendance = require('../models/attendance');
var moment = require('moment');
var Project = require('../models/project');
var PerformanceAppraisal = require('../models/performance_appraisal');
var flash = require('connect-flash');
var csrf = require('csrf');
var csrfProtection = csrf();

router.use('/', isLoggedIn, function checkAuthentication(req, res, next) {
    next();
});

router.get('/', function viewHomePage(req, res, next) {

    res.render('Manager/managerHome', {
        title: 'Manager Home',
        csrfToken: req.csrfToken(),
    });
}

```

КПІ.IT-8427.045440.03.13					Арк.
Змін.	Арк.	№ докум.	Підп.	Дата.	96

```

        userName: req.session.user.name
    });
});

router.get('/view-employees', function viewEmployees(req, res) {

    var userChunks = [];
    var chunkSize = 3;
    if (req.user.type === 'project_manager') {
        //find is asynchronous function
        User.find({type: 'employee'}).sort({_id: -1}).exec(function getUser(err,
docs) {
            for (var i = 0; i < docs.length; i++) {
                userChunks.push(docs[i]);
            }
            res.render('Manager/viewemp_project', {
                title: 'List Of Employees',
                csrfToken: req.csrfToken(),
                users: userChunks, errors: 0, userName: req.session.user.name
            });
        });
    }
    else if (req.user.type === 'accounts_manager') {
        //find is asynchronous function
        var salaryChunks = [];

        User.find({$or: [{type: 'employee'}, {type:
'project_manager'}]}).sort({_id: -1}).exec(function getUser(err, docs) {
            if (err) {
                console.log(err);
            }
            for (var i = 0; i < docs.length; i++) {
                userChunks.push(docs[i]);
            }
        });

        setTimeout(getUserSalaries, 900);

        function getUserSalaries() {

            function callback(i) {
                if (i < userChunks.length) {
                    UserSalary.find({employeeID: userChunks[i]._id}, function
(err, salary) {
                        console.log(i);

                        if (err) {
                            console.log(err);
                        }
                        if (salary.length > 0) {
                            salaryChunks.push(salary[0]);
                        }
                        else {
                            var newSalary = new UserSalary();
                            newSalary.accountManagerID = req.session.user._id;

```

```

        newSalary.employeeID = userChunks[i]._id;
        newSalary.save(function (err) {
            if (err) {
                console.log(err);
            }
            salaryChunks.push(newSalary);
        })
    }

    callback(i + 1);
});
}

callback(0);
}

setTimeout(render_view, 2000);
function render_view() {

    res.render('Manager/viewemp_accountant', {
        title: 'List Of Employees', csrfToken: req.csrfToken(),
        users: userChunks, salary: salaryChunks, userName:
req.session.user.name
    });

}

}

});

router.get('/all-employee-skills/:id', function viewAllEmployeeSkills(req, res,
next) {

    var employeeId = req.params.id;
    User.findById(employeeId, function getUser(err, user) {
        if (err) {
            console.log(err);
        }
        res.render('Manager/employeeSkills', {
            title: 'List Of Employee Skills',
            employee: user,
            moment: moment,
            csrfToken: req.csrfToken(),
            userName: req.session.user.name
        });
    });
});

router.get('/all-employee-projects/:id', function viewAllEmployeeProjects(req,
res, next) {

```



```

var employeeId = req.params.id;
var projectChunks = [];

//find is asynchronous function
Project.find({employeeID: employeeId}).sort({_id: -1}).exec(function
getProject(err, docs) {
    var hasProject = 0;
    if (docs.length > 0) {
        hasProject = 1;
    }
    for (var i = 0; i < docs.length; i++) {
        projectChunks.push(docs[i]);
    }
    User.findById(employeeId, function getUser(err, user) {
        if (err) {
            console.log(err);
        }
        res.render('Manager/employeeAllProjects', {
            title: 'List Of Employee Projects',
            hasProject: hasProject,
            projects: projectChunks,
            csrfToken: req.csrfToken(),
            user: user,
            userName: req.session.user.name
        });
    });
});

});

router.get('/employee-project-info/:id', function viewEmployeeProjectInfo(req,
res, next) {

    var projectId = req.params.id;
    Project.findById(projectId, function getProject(err, project) {
        if (err) {
            console.log(err);
        }
        User.findById(project.employeeID, function getUser(err, user) {
            if (err) {
                console.log(err);
            }
            res.render('Manager/projectInfo', {
                title: 'Employee Project Information',
                project: project,
                employee: user,
                moment: moment,
                csrfToken: req.csrfToken(),
                message: '',
                userName: req.session.user.name
            });
        });
    });

});

});

```

```

router.get('/provide-performance-appraisal/:id', function
providePerformanceAppraisal(req, res, next) {

    var employeeId = req.params.id;
    var userChunks = [];
    PerformanceAppraisal.find({employeeID: employeeId}, function
getPerformanceAppraisal(err, pa) {
        if (pa.length > 0) {
            User.find({type: 'employee'}, function getUser(err, docs) {
                for (var i = 0; i < docs.length; i++) {
                    userChunks.push(docs[i]);
                }
                res.render('Manager/viewemp_project', {
                    title: 'List Of Employees',
                    csrfToken: req.csrfToken(),
                    users: userChunks, errors: 1,
                    userName: req.session.user.name
                });
            });
        }
        else {
            User.findById(employeeId, function getUser(err, user) {

                if (err) {
                    console.log(err);
                }
                res.render('Manager/performance_appraisal', {
                    title: 'Provide Performance Appraisal',
                    csrfToken: req.csrfToken(),
                    employee: user,
                    moment: moment,
                    message: '',
                    userName: req.session.user.name
                });
            });
        }
    })

});

router.get('/view-attendance-current', function viewCurrentMarkedAttendance(req,
res, next) {

    var attendanceChunks = [];

    Attendance.find({
        employeeID: req.user._id,
        month: new Date().getMonth() + 1,
        year: new Date().getFullYear()
    }).sort({_id: -1}).exec(function getAttendanceSheet(err, docs) {
        var found = 0;
        if (docs.length > 0) {
            found = 1;
        }
    })
});

```

КПІ.IT-8427.045440.03.13					Арк.
Змін.	Арк.	№ докум.	Підп.	Дата.	100

```

        for (var i = 0; i < docs.length; i++) {
            attendanceChunks.push(docs[i]);
        }
        res.render('Manager/viewAttendance', {
            title: 'Attendance Sheet',
            month: new Date().getMonth() + 1,
            csrfToken: req.csrfToken(),
            found: found,
            attendance: attendanceChunks,
            moment: moment,
            userName: req.session.user.name
        });
    });
});

router.get('/apply-for-leave', function applyForLeave(req, res, next) {

    res.render('Manager/managerApplyForLeave', {
        title: 'Apply for Leave',
        csrfToken: req.csrfToken(),
        userName: req.session.user.name
    });
});

router.get('/applied-leaves', function appliedLeaves(req, res, next) {

    var leaveChunks = [];

    //find is asynchronous function
    Leave.find({applicantID: req.user._id}).sort({_id: -1}).exec(function
    getLeave(err, docs) {
        var hasLeave = 0;
        if (docs.length > 0) {
            hasLeave = 1;
        }
        for (var i = 0; i < docs.length; i++) {
            leaveChunks.push(docs[i]);
        }

        res.render('Manager/managerAppliedLeaves', {
            title: 'List Of Applied Leaves',
            csrfToken: req.csrfToken(),
            hasLeave: hasLeave,
            leaves: leaveChunks,
            userName: req.session.user.name
        });
    });
});

router.get('/view-profile', function viewProfile(req, res, next) {

    User.findById(req.user._id, function getUser(err, user) {
        if (err) {
            console.log(err);
        }
        res.render('Manager/viewManagerProfile', {

```

```

        title: 'Profile',
        csrfToken: req.csrfToken(),
        employee: user,
        moment: moment,
        userName: req.session.user.name
    });
});

});

router.get('/view-project/:project_id', function viewProject(req, res, next) {

    var projectId = req.params.project_id;
    Project.findById(projectId, function getProject(err, project) {
        if (err) {
            console.log(err);
        }
        res.render('Manager/viewManagerProject', {
            title: 'Project Details',
            project: project,
            csrfToken: req.csrfToken(),
            moment: moment,
            userName: req.session.user.name
        });
    });

});

});

router.get('/view-all-personal-projects', function viewAllPersonalProjects(req,
res, next) {

    var projectChunks = [];
    Project.find({employeeID: req.user._id}).sort({_id: -1}).exec(function
getProject(err, docs) {
        var hasProject = 0;
        if (docs.length > 0) {
            hasProject = 1;
        }
        for (var i = 0; i < docs.length; i++) {
            projectChunks.push(docs[i]);
        }
        res.render('Manager/viewManagerPersonalProjects', {
            title: 'List Of Projects',
            hasProject: hasProject,
            projects: projectChunks,
            csrfToken: req.csrfToken(),
            userName: req.session.user.name
        });
    });

});

});

router.get('/generate-pay-slip/:employee_id', function generatePaySlip(req, res,
next) {

```

					КПІ.IT-8427.045440.03.13	Арк.
Змін.	Арк.	№ докум.	Підп.	Дата.		
						102

```

var employeeId = req.params.employee_id;
User.findById(employeeId, function getUser(err, user) {
  if (err) {
    console.log(err);
  }
  PaySlip.find({employeeID: employeeId}, function getPaySlip(err, docs) {

    var pay_slip;
    var hasPaySlip = 0;
    if (docs.length > 0) {
      hasPaySlip = 1;
      pay_slip = docs[0];
    }
    else {
      var newPS = new PaySlip();
      newPS.accountManagerID = req.user._id;
      newPS.employeeID = employeeId;
      newPS.bankName = 'abc';
      newPS.branchAddress = 'abc';
      newPS.basicPay = 0;
      newPS.overtime = 0;
      newPS.conveyanceAllowance = 0;

      newPS.save(function savePaySlip(err) {
        if (err) {
          console.log(err);
        }
        pay_slip = newPS;
      })
    }
  })

  setTimeout(render_view, 900)
  function render_view() {
    res.render('Manager/generatePaySlip', {
      title: 'Generate Pay Slip',
      csrfToken: req.csrfToken(),
      employee: user,
      pay_slip: pay_slip,
      moment: moment,
      hasPaySlip: hasPaySlip
      , userName: req.session.user.name
    });
  }
});

});

});

router.post('/apply-for-leave', function applyForLeave(req, res, next) {

  var newLeave = new Leave();
  newLeave.applicantID = req.user._id;
  newLeave.title = req.body.title;
  newLeave.type = req.body.type;

```

```

newLeave.startDate = new Date(req.body.start_date);
newLeave.endDate = new Date(req.body.end_date);
newLeave.period = req.body.period;
newLeave.reason = req.body.reason;
newLeave.appliedDate = new Date();
newLeave.adminResponse = 'Pending';
newLeave.save(function saveLeave(err) {
    if (err) {
        console.log(err);
    }
    res.redirect('/manager/applied-leaves');
});

});

router.post('/set-bonus', function setBonus(req, res) {

    UserSalary.findOne({employeeID: req.body.employee_bonus}, function
getUser(err, us) {
        if (err) {
            console.log(err)
        }
        us.bonus = req.body.bonus;
        us.reason = req.body.reason;
        us.save(function saveUserSalary(err) {
            if (err) {
                console.log(err);
            }
            res.redirect('/manager/view-employees')
        })
    })

})

router.post('/set-salary', function setSalary(req, res) {

    var employee_id = req.body.employee_salary;
    UserSalary.findOne({employeeID: employee_id}, function (err, us) {
        if (err) {
            console.log(err)
        }
        console.log(us);
        us.salary = Number(req.body.salary);
        us.save(function setUserSalary(err) {
            if (err) {
                console.log(err);
            }
            res.redirect('/manager/view-employees')
        })
    })

})

router.post('/increment-salary', function incrementSalary(req, res) {

    UserSalary.findOne({employeeID: req.body.employee_increment}, function
getUserSalary(err, us) {

```

```

        if (err) {
            console.log(err)
        }
        us.salary = Number(req.body.current_salary) +
        Number(req.body.amount_increment);
        us.save(function saveUserSalary(err) {
            if (err) {
                console.log(err);
            }
            res.redirect('/manager/view-employees')
        })
    })
})

router.post('/provide-performance-appraisal', function
providePerformanceAppraisal(req, res) {

    var employeeId = req.body.employee_id;
    var newPerformanceAppraisal = new PerformanceAppraisal();
    newPerformanceAppraisal.employeeID = employeeId;
    newPerformanceAppraisal.projectManagerID = req.user._id;
    newPerformanceAppraisal.rating = req.body.performance_rating;
    newPerformanceAppraisal.positionExpertise = req.body.expertise;
    newPerformanceAppraisal.approachTowardsQualityOfWork =
req.body.approach_quality;
    newPerformanceAppraisal.approachTowardsQuantityOfWork =
req.body.approach_quantity;
    newPerformanceAppraisal.leadershipManagementSkills = req.body.lead_manage;
    newPerformanceAppraisal.communicationSkills = req.body.skills_com;
    newPerformanceAppraisal.commentsOnOverallPerformance = req.body.comments;
    newPerformanceAppraisal.save(function savePerformanceAppraisal(err) {
        if (err) {
            console.log(err);
        }
        res.redirect('/manager/view-employees');
    });
});

router.post('/generate-pay-slip', function generatePaySlip(req, res) {

    var employeeId = req.body.employee_id;
    PaySlip.find({employeeID: employeeId}, function getPaySlip(err, docs) {
        if (err) {
            console.log(err);
        }
        docs[0].bankName = req.body.bname;
        docs[0].branchAddress = req.body.baddress;
        docs[0].basicPay = req.body.pay;
        docs[0].overtime = req.body.otime;
        docs[0].conveyanceAllowance = req.body.allowance;
        docs[0].save(function savePaySlip(err) {
            if (err) {
                console.log(err);
            }
            res.redirect('/manager/view-employees');
        })
    })
});

```

```

    })
  })
});

router.post('/view-attendance', function viewAttendance(req, res, next) {

  var attendanceChunks = [];
  Attendance.find({
    employeeID: req.user._id,
    month: req.body.month,
    year: req.body.year
  }).sort({_id: -1}).exec(function getAttendanceSheet(err, docs) {
    var found = 0;
    if (docs.length > 0) {
      found = 1;
    }
    for (var i = 0; i < docs.length; i++) {
      attendanceChunks.push(docs[i]);
    }
    res.render('Manager/viewAttendance', {
      title: 'Attendance Sheet',
      month: req.body.month,
      csrfToken: req.csrfToken(),
      found: found,
      attendance: attendanceChunks,
      moment: moment,
      userName: req.session.user.name
    });
  });
});

});

router.post('/mark-manager-attendance', function markAttendance(req, res, next) {

  Attendance.find({
    employeeID: req.user._id,
    date: new Date().getDate(),
    month: new Date().getMonth() + 1,
    year: new Date().getFullYear()
  }, function getAttendance(err, docs) {
    var found = 0;
    if (docs.length > 0) {
      found = 1;
    }
    else {

      var newAttendance = new Attendance();
      newAttendance.employeeID = req.user._id;
      newAttendance.year = new Date().getFullYear();
      newAttendance.month = new Date().getMonth() + 1;
      newAttendance.date = new Date().getDate();
      newAttendance.present = 1;
      newAttendance.save(function saveAttendance(err) {
        if (err) {
          console.log(err);
        }
      });
    }
  });
});

```



```

        }

    });
}
res.redirect('/manager/view-attendance-current');

});

});
module.exports = router;

function isLoggedIn(req, res, next) {

    if (req.isAuthenticated()) {
        return next();
    }
    res.redirect('/');
}

```

					КПІ.ІТ-8427.045440.03.13	Арк.
Змін.	Арк.	№ докум.	Підп.	Дата.		
						107

Факультет інформатики та обчислювальної техніки
Кафедра інформатики та програмної інженерії

“ЗАТВЕРДЖЕНО”

Завідувач кафедри

_____ Едуард ЖАРІКОВ

“ ____ ” _____ 2022 р.

**ПОРТАЛ ОСОБИСТОГО КАБІНЕТУ СПІВРОБІТНИКА СУЧАСНОЇ
КОМПАНІЇ**

Програма та методика тестування

КПІ.IT-8427.045440.04.51

“ПОГОДЖЕНО”

Керівник проєкту:

_____ Тетяна ЛІХОУЗОВА

Нормоконтроль:

_____ Катерина ЛІЩУК

Виконавець:

_____ Світлана ТРОФІМОВА

Київ – 2022

ЗМІСТ

1 Об'єкт випробувань	3
2 Мета тестування	4
3 Методи тестування.....	5
4 Засоби та порядок тестування.....	6

					КПІ.ІТ-8427.045440.04.51	Арк. 109
Змін.	Арк.	№ докум.	Підп.	Дата.		

1 ОБ'ЄКТ ВИПРОБУВАНЬ

Об'єктом випробування є веб-застосунок для керування людськими ресурсами HR-портал. Програмний продукт розроблений на мові Node.js та за допомогою бази даних MongoDB.

					КПІ.ІТ-8427.045440.04.51	Арк. 110
Змін.	Арк.	№ докум.	Підп.	Дата.		

2 МЕТА ТЕСТУВАННЯ

Метою тестування є наступне:

- перевірка правильності роботи програмного забезпечення у відповідно до функціональних вимог;
- знаходження проблем та недоліків з метою їх усунення;
- перевірка зручності графічного інтерфейсу.

					КПІ.ІТ-8427.045440.04.51	Арк. 111
Змін.	Арк.	№ докум.	Підп.	Дата.		

3 МЕТОДИ ТЕСТУВАННЯ

Для тестування програмного забезпечення використовуються такі методи:

- статичне тестування – перевіряється уся документація, яка аналізується на предмет дотримання стандартів програмування;
- динамічне тестування – застосовується в процесі виконання програми. Коректність програмного засобу перевіряється на безлічі тестів. При прогоні кожного з них збираються та аналізуються дані про проблеми в роботі програми;
- тестування «білої скриньки» – об'єктом тестування тут є внутрішня поведінка програми. Перевіряється коректність побудови всіх елементів програми та правильність їхньої взаємодії один з одним.

					КПІ.IT-8427.045440.04.51	Арк. 112
Змін.	Арк.	№ докум.	Підп.	Дата.		

4 ЗАСОБИ ТА ПОРЯДОК ТЕСТУВАННЯ

Тестування виконується вручну, з метою знаходження помилок та недоліків як у функціональній частині програмного забезпечення так і в зручності в користуванні. Для того, щоб перевірити працездатність та відмовостійкість додатку, необхідно провести наступні тестування:

- динамічне тестування на відповідність функціональним вимогам;
- тестування на мобільних пристроях з різною роздільною здатністю екрану;
- тестування на мобільних пристроях з різною версією операційної системи;
- тестування на виведення повідомлень про помилку, коли це необхідно;
- тестування зміни орієнтації екрану;
- тестування працездатності програми у випадку відсутності з'єднання до мережі;
- тестування інтерфейсу користувача;
- тестування зручності використання.

Факультет інформатики та обчислювальної техніки
Кафедра інформатики та програмної інженерії

“ЗАТВЕРДЖЕНО”

Завідувач кафедри

_____ Едуард ЖАРІКОВ

“ ____ ” _____ 2022 р.

**ПОРТАЛ ОСОБИСТОГО КАБІНЕТУ СПІВРОБІТНИКА СУЧАСНОЇ
КОМПАНІЇ**

Керівництво користувача

КПІ.ІТ-8427.045440.05.34

“ПОГОДЖЕНО”

Керівник проєкту:

_____ Тетяна ЛІХОУЗОВА

Нормоконтроль:

_____ Катерина ЛІЩУК

Виконавець:

_____ Світлана ТРОФІМОВА

Київ – 2022

ЗМІСТ

1 Загальні відомості	3
2 Підготовка до роботи.....	4
2.1 Системні вимоги для коректної роботи	4
2.2 Завантаження застосунку.....	4
2.3 Перевірка коректної роботи.....	4
3 Робота із застосунком.....	5

					КПІ.ІТ-8427.045440.04.51	Арк. 2
Змін.	Арк.	№ докум.	Підп.	Дата.		

1 ЗАГАЛЬНІ ВІДОМОСТІ

«HR-портал» - це веб-застосунок, який призначений для управління персоналом компанії.

Це веб-додаток, який є повною системою управління людськими ресурсами, яка зберігає інформацію про людські ресурси, освіту всіх співробітників, їхній останній досвід, відвідуваність, управління відпустками та їхні поточні виділені проекти. Цей проект побудований на Node Express, який є фреймворком веб-додатків для Node.js, а інтерфейс створений за допомогою HTML, Bootstrap, CSS та JS.

					КПІ.IT-8427.045440.04.51	Арк.
Змін.	Арк.	№ докум.	Підп.	Дата.		3

1 ПІДГОТОВКА ДО РОБОТИ

1.1 Системні вимоги для коректної роботи

Для успішної роботи даного застосунку необхідне виконання наступних вимог:

- наявність ПК з операційною системою на базі Windows/Ubuntu/macOS;
- встановлена остання версія node;
- для встановлення додатку на пристрої повинно бути не менше 70 МБ вільної пам'яті.

1.2 Завантаження застосунку

На даний момент застосунок можна встановити власноруч, використовуючи відповідний zip-файл. Для цього спершу необхідно завантажити його на пристрій, а потім, використовуючи який-небудь інсталятор, виконати встановлення даного додатку.

Для успішної роботи застосунку необхідно виконати декілька не складних дій. По-перше, необхідно встановити недостаючі бібліотеки шляхом відкриття у терміналі папки проєкту та запуску `npm install`. Після цього необхідно встановити підключення до MongoDB та виконати файл `seeders`, який встановить мінімальні початкові дані та створить необхідні схеми для зберігання даних.

1.3 Перевірка коректної роботи

Після встановлення додатку необхідно запустити файл `bin/www`, який є завантажувачем серверу. Необхідно відкрити будь-який браузер та ввести у адресному рядку `localhost:3000`, після цього браузер буде переправлено на домашню сторінку застосунку.

					КПІ.IT-8427.045440.04.51	Арк.
Змін.	Арк.	№ докум.	Підп.	Дата.		4

2 РОБОТА ІЗ ЗАСТОСУНКОМ

При запуску програмного застосунку користувачу буде відображено два поля для вводу логіну та паролю для входу у систему (рис. 2.1).

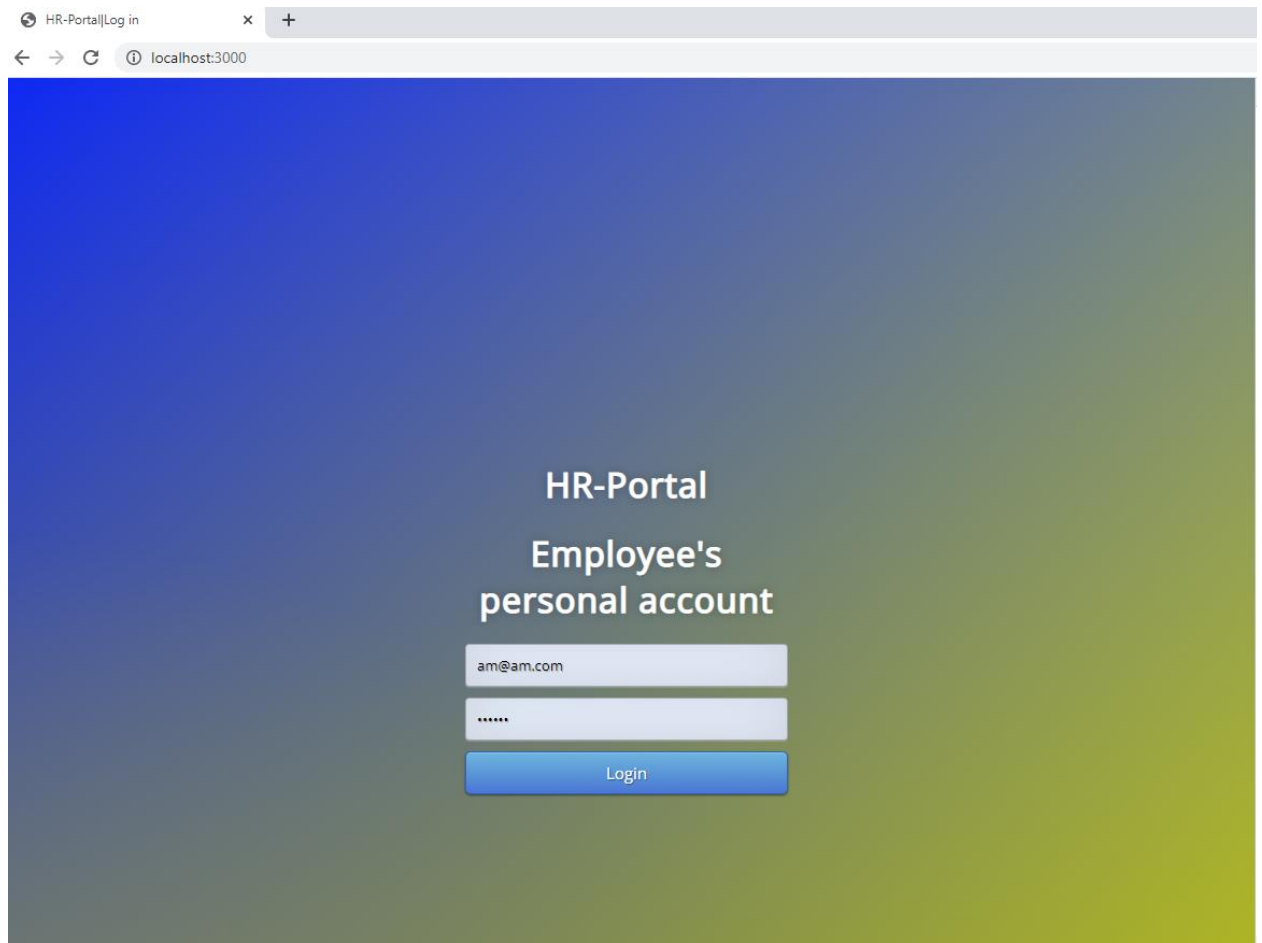


Рисунок 2.1 – Початкова сторінка додатку

Користувач має змогу ввести свій логін та пароль. Система підтримує сервіс збереження паролів від гугл.

Після введення коректних даних відбудеться перехід на домашню сторінку додатку (рис. 2.2).

					КПІ.ІТ-8427.045440.04.51	Арк.
Змін.	Арк.	№ докум.	Підп.	Дата.		5

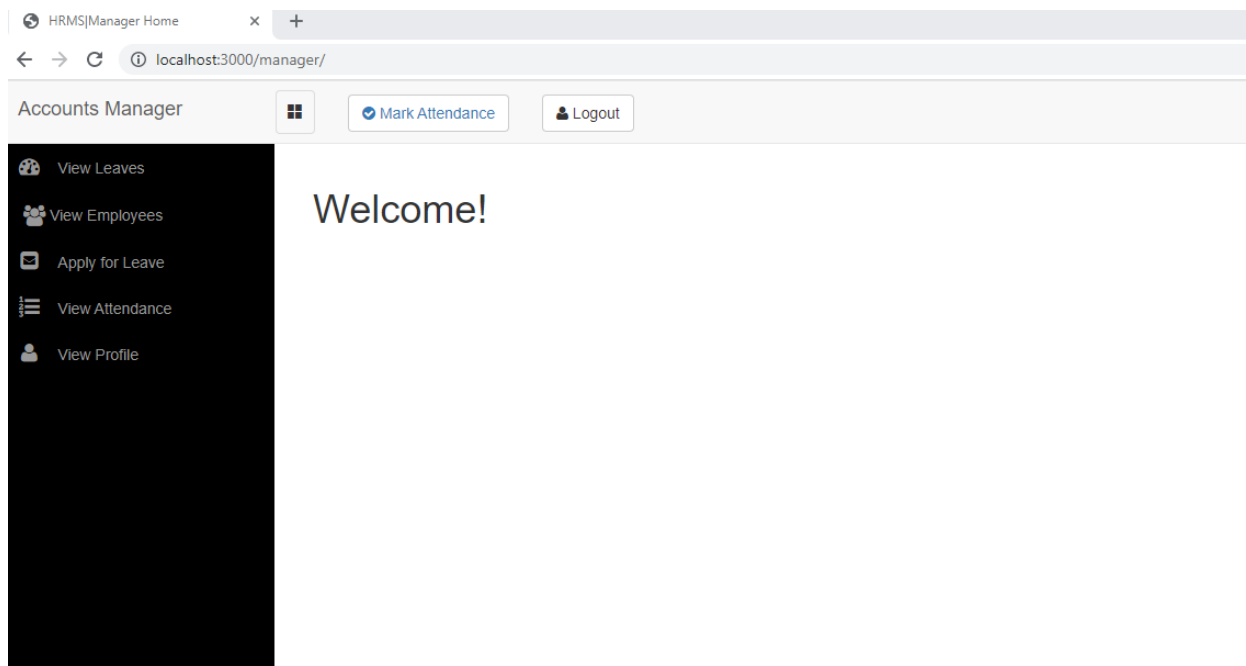


Рисунок 2.2 – Початкова сторінка додатку

Сторінка створення запиту на відпустку доступна по клавiшi “Apply for Leave”. Після натиснення відбувається перехiд до сторiнки створення запиту (рис. 2.3).

Рисунок 2.3 – сторінка запиту на відпустку

Користувач повинен ввести коректні дані згідно з його побажання відпустки (рис. 2.4).

					КПІ.IT-8427.045440.04.51	Арк. 6
Змін.	Арк.	№ докум.	Підп.	Дата.		

Accounts Manager

View Leaves
View Employees
Apply for Leave
View Attendance
View Profile

Mark Attendance Logout

Application Details

Title:
Vacation Request

Type:
paid

Period of Leave:
5

Start Date:
27.06.2022

End Date:
01.07.2022

Reason for Leave:
Need to have a resf

Cancel Apply

Рисунок 2.4 – Введено коректні дані

Після цього необхідно натиснути кнопку “Apply” після чого запит буде успішно створений. Та піде на погодження до адміністратора.

Одним з функціоналом для працівника є відмітка присутності, яку можна отримати за допомогою кнопки у таск-барі (рис. 2.5).

Accounts Manager

Mark Attendance Logout

Mark Attendance

Are you sure you want to mark attendance.

No Yes

Рисунок 2.5 – відмітка присутності

Після натиснення кнопки ми отримаємо повідомлення, яке уточнює чи справді запит необхідно виконати.

Після цього можна перевірити свої дані про присутність за період на сторінці “View Attendance” (рис. 2.6).

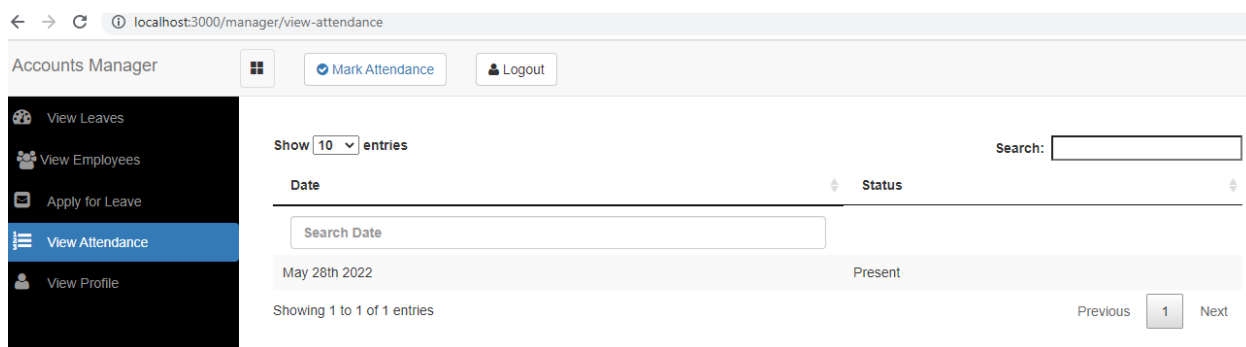


Рисунок 2.6 – Таблиця присутності

Користувач має можливість переглянути свої дані натиснувши кнопку “View Profile”

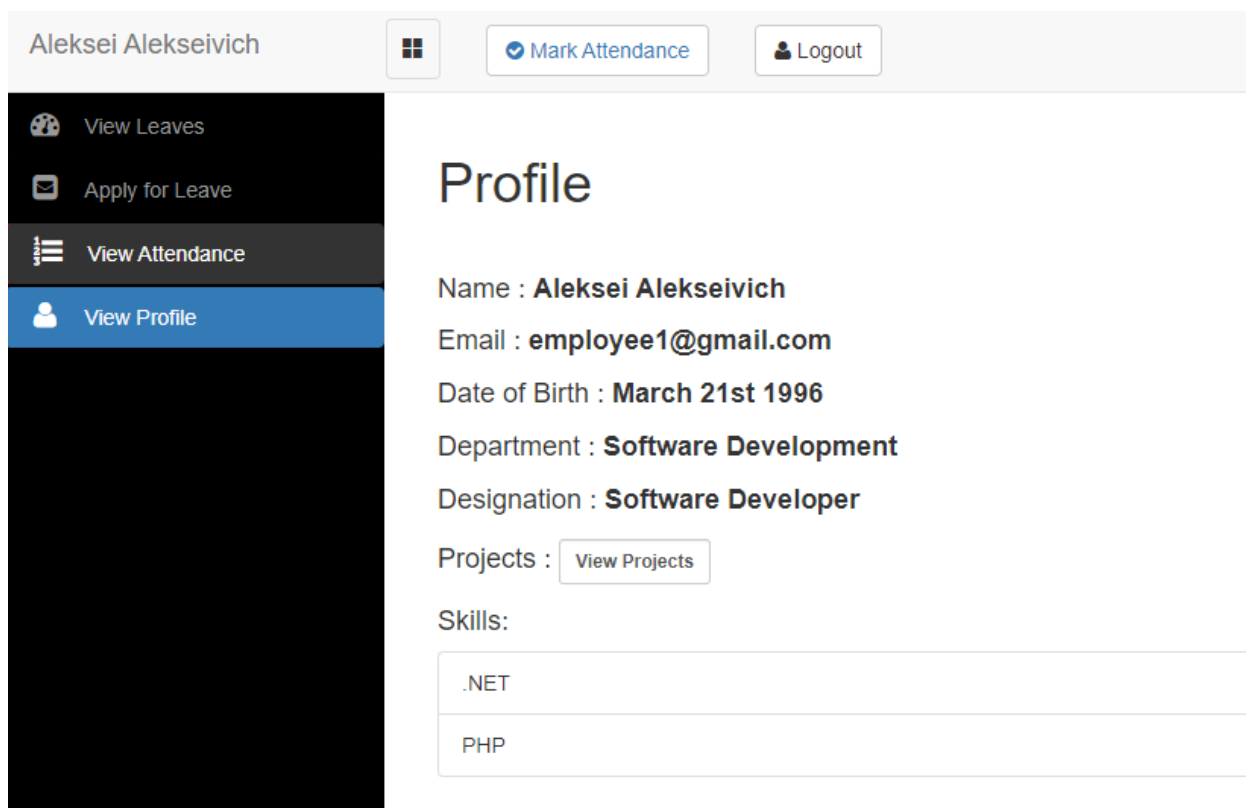


Рисунок 2.7 – Дані профіля

Відповідно адміністратор системи має можливість додавати нових користувачів

Trofimova Svitlana

Mark Attendance

Logout

Employees

View Employees

Add Employees

Leave Applications

View Attendance

View Profile

Employee Details

Name:

Diana Obiedkova

Email Address:

diana@hrms.com

Date of Birth:

31.10.2001

Password:

Contact Number:

+380507404654

Department:

Human Resource

Skills:

N/A

Designation:

Human Resource Manager

Cancel

Register

Рисунок 2.8 – Дані профіля

Після введення даних необхідно натиснути кнопку реєстрація та можемо переглянути доданого працівника (рис. 2.9).

Trofimova Svitlana

Mark Attendance

Logout

Employees

View Employees

Add Employees

Leave Applications

View Attendance

View Profile

All Employees

Show 10 entries

Search:

Name	Email	Contact No.	Department	Designation	
Search Nar	Search Email	Search Contac	Search Departr	Search Designatic	Search
Diana Obiedkova	diana@hrms.com	+380507404654	Human Resource	Human Resource Manager	
Aleksei Aleksevich	employee1@gmail.com	+380(073)485-745-55	Software Development	Software Developer	
sample	sample@sample.com	0333-4552191	Human Resource	Accounts Manager	
Accounts Manager	am@am.com	0300-4814710	N/A	N/A	
Project manager	pm@pm.com	0333-4552191	N/A	N/A	

Showing 1 to 5 of 5 entries

Previous

1

Next

Рисунок 2.9 – Доданий працівник

За допомогою кнопок у таблиці можна використовувати різні операції з даними: корегувати, переглядати, видаляти дані про працівника, а також про проєкт (рис. 2.10).



Рисунок 2.10 – Кнопки керування даними

Після завершення роботи з системою користувач має змогу вийти із системи та змінити профіль або закінчити роботу. Кнопка виходу представлена на таск барі та зображена на рисунку 2.11.

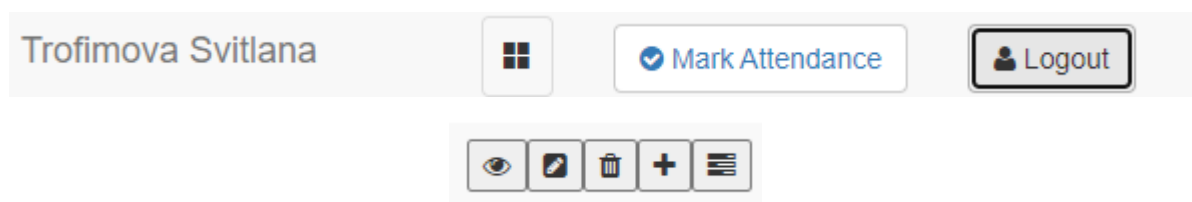


Рисунок 2.11 – Кнопка виходу із системи

Факультет інформатики та обчислювальної техніки
Кафедра інформатики та програмної інженерії

“ЗАТВЕРДЖЕНО”

Завідувач кафедри

_____ Едуард ЖАРІКОВ

“ ____ ” _____ 2022 р.

**ПОРТАЛ ОСОБИСТОГО КАБІНЕТУ СПІВРОБІТНИКА СУЧАСНОЇ
КОМПАНІЇ**

Графічний матеріал

КПІ.ІТ-8427.045440.05.99

“ПОГОДЖЕНО”

Керівник проєкту:

_____ Тетяна ЛІХОУЗОВА

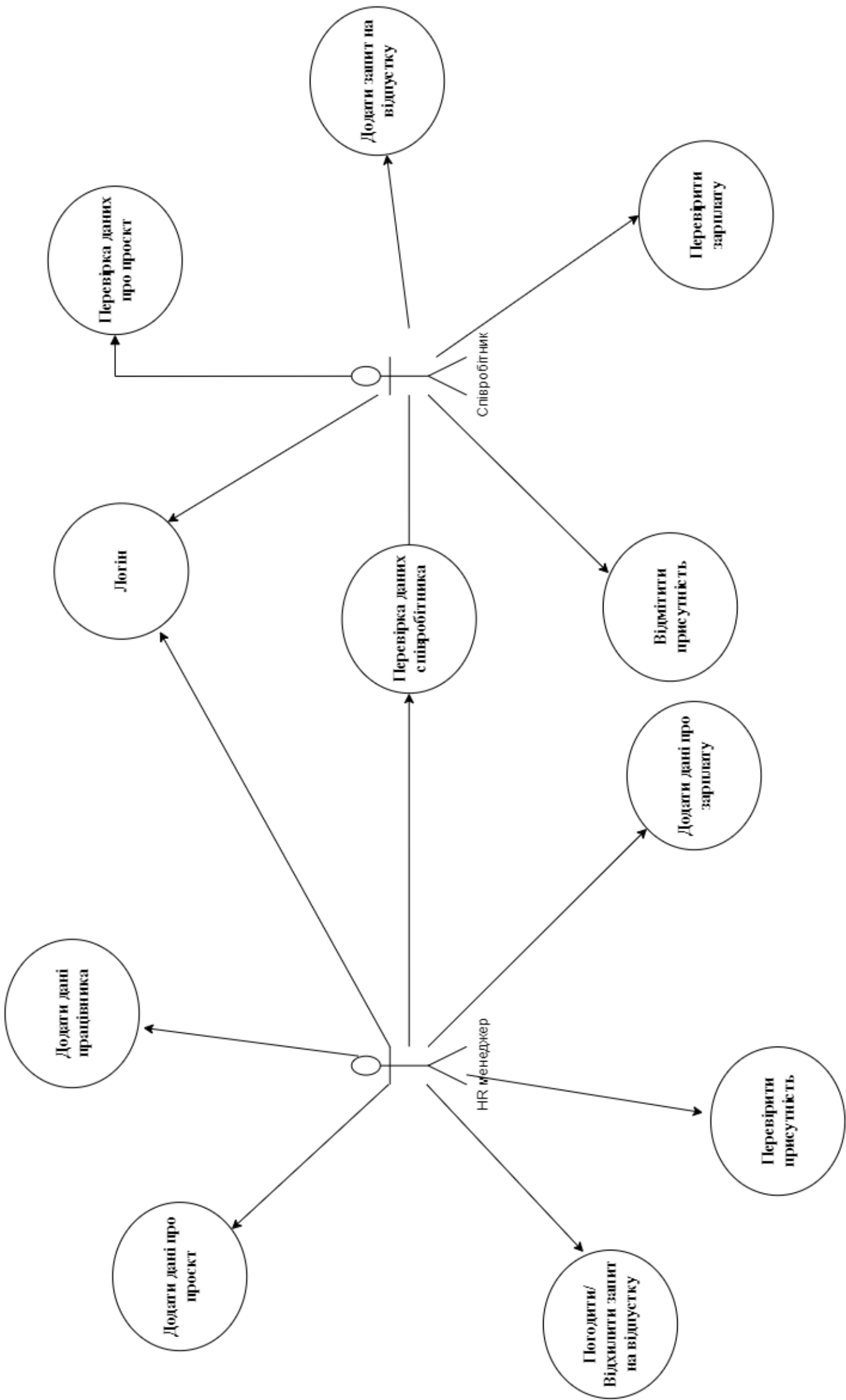
Нормоконтроль:

_____ Катерина ЛІЩУК

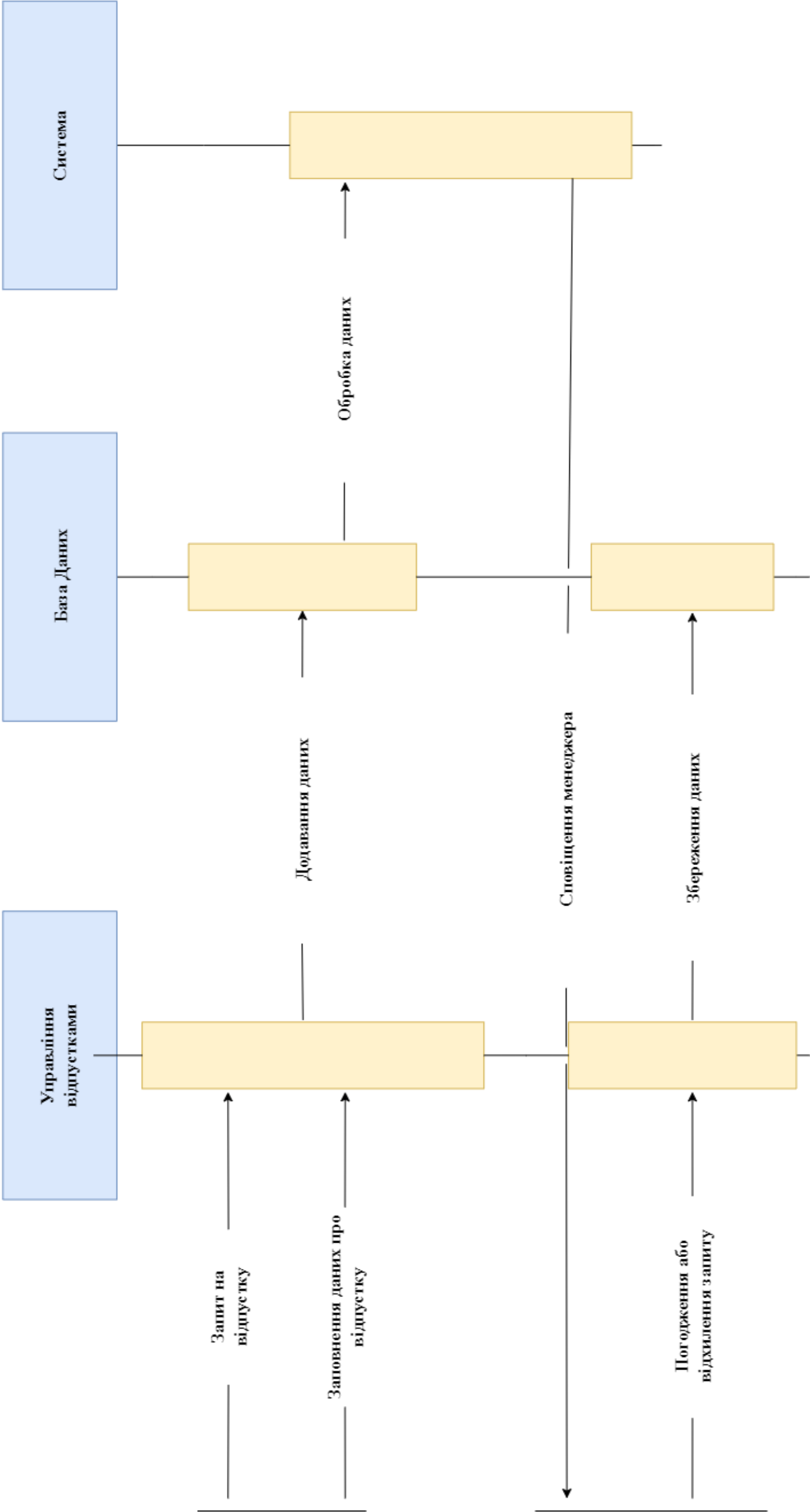
Виконавець:

_____ Світлана ТРОФІМОВА

Київ – 2022

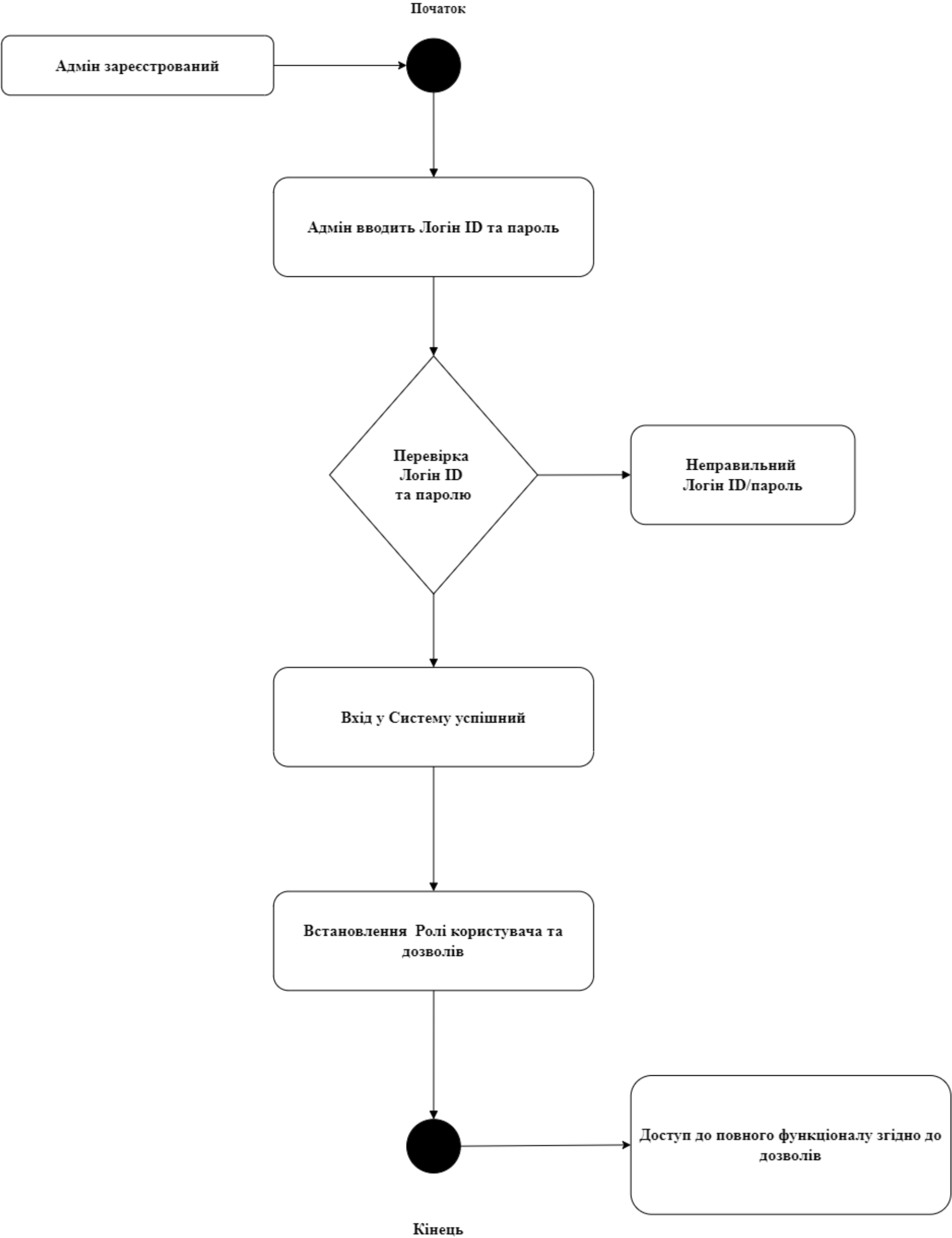


					КПІ.ІТ-8427.045440.06.99					
					Схема структурна варіантів використання HR-порталу	Лит.			Арк.	Аркушіє
Зм.	Арк.	№ докум.	Підп.	Дата						1
Розроб.	Трофімова С.С									
Перев.	Ліхоузова Т.А.									
Т. Кон.						Аркуш			Аркушіє	
					Портал особистого кабінету співробітника сучасної компанії	КПІ ім.Ігоря Сікорського Кафедра ІПІ гр. ІТ-84				
Н. Кон.	Ліщук К.І.									
Затв.	Ліхоузова Т.А.									



Підп. дата	Інв. № дубл.	Інв. № взаєм. підп.	Підп. дата	Інв. № підп.	

					КПІ.ІТ-8427.045440.07.99					
					Схема взаємодії програми при вході у систему	Лист.		Арк.	Аркуші	
									1	
						Аркуш		Аркуші		
						КПІ ім.Ігоря Сікорського Кафедра ІПІ гр. ІТ-84				
Зм.	Арк.	№ докум.	Підп.	Дата						
	Розроб.	Трофімова С.С								
	Перев.	Ліхоузова Т.А.								
	Т. Кон.									
	Н. Кон.	Ліщук К.І.			Портал особистого кабінету співробітника сучасної компанії					
	Затв.	Ліхоузова Т.А.								



Інв. № підп.	Підп. дата	Інв. № взаєм. підп.	Інв. № дубл.	Підп. дата

					КПІ.ІТ-8427.045440.08.99				
					Схема взаємодії програми при запиті на відпустку	Лит.		Арк.	Аркуші
Зм.	Арк.	№ докум.	Підп.	Дата					1
Розроб.	Трофімова С.С								
Перев.	Ліхоузова Т.А.								
Т. Кон.									
						Аркуш		Аркуші	
Н. Кон.	Ліщук К.І.				Портал особистого кабінету співробітника сучасної компанії	КПІ ім.Ігоря Сікорського Кафедра ІПІ гр. ІТ-84			
Затв.	Ліхоузова Т.А.								

Имя пользователя:
Лісовиченко Олег Іванович

ID проверки:
1011544499

Дата проверки:
11.06.2022 07:04:35 EEST

Тип проверки:
Doc vs Internet + Library

Дата отчета:
11.06.2022 07:04:57 EEST

ID пользователя:
76913

Название файла: IT-84_Трофімова_ПЗ (2)

Количество страниц: 53 Количество слов: 8754 Количество символов: 67071 Размер файла: 779.08 KB ID файла: 1011416752

9.73% Совпадения

Наибольшее совпадение: 3.32% с источником из Библиотеки (ID файла: 1011388137)

0.78% Источники из Интернета

4

Страница 55

9.29% Источники из Библиотеки

170

Страница 55

0% Цитат

Исключение цитат выключено

Исключение списка библиографических ссылок выключено

0% Исключений

Нет исключенных источников

Модификации

Обнаружены модификации текста. Подробная информация доступна в онлайн-отчете.

Замененные символы

4